Transcript Retrieval & Chatbot Optimization for Meadow9+

Interim Report

Toh Kar Le
U2221258J

Supervisor: Dr Chng Eng Siong

College of Computing and Data Science
Nanyang Technological University

Academic Year 2025/2026

Abstract

Long-duration meeting transcription systems have become increasingly prevalent in both enterprise and professional settings. While speech-to-text accuracy has improved substantially, effective and efficient retrieval of information from lengthy transcripts remains a significant challenge. Meadow9+, a long-meeting transcription application developed by ABAX AI, recently introduced a chatbot feature to address this need; however, its current implementation processes entire transcripts directly as input to a large language model (LLM), resulting in high latency, excessive computational cost, and degraded answer quality due to long-context limitations.

This project proposes and develops a Retrieval-Augmented Generation (RAG)–based system to optimize transcript retrieval and chatbot performance for Meadow9+. By embedding transcript segments into a vector space and retrieving only the most relevant contextual chunks for each user query, the proposed system significantly reduces inference overhead while preserving answer accuracy. In addition, this work introduces a comprehensive evaluation pipeline that systematically assesses both retrieval quality and end-to-end question-answering performance using metrics such as Hit Rate, Mean Reciprocal Rank (MRR), F1 score, and overall accuracy, alongside detailed latency benchmarking.

Experimental results demonstrate that the RAG-based chatbot achieves comparable accuracy to the existing full-transcript LLM approach while delivering substantial improvements in response time, validating the effectiveness of retrieval-driven optimization. Beyond immediate chatbot acceleration, the embedding-based architecture enables extensibility toward more advanced retrieval techniques, including phonetic search, fuzzy matching, and hybrid retrieval strategies tailored to noisy speech-to-text outputs. This report presents the system design, implementation progress, and evaluation results to date, and outlines future work focused on building a comprehensive, efficient, and deployable retrieval system.

Contents

Abstract **1**

1 Introduction **8**
1.1 Background . . . 8
1.2 Motivations and Objectives . . . 8
1.3 Scope . . . 9
1.4 Contributions . . . 10
1.5 Project Schedule . . . 11

2 Literature Review **13**
2.1 Otter.ai . . . 13
2.1.1 Retrieval Strategies . . . 13
2.1.2 Accuracy and Latency Characteristics . . . 15
2.1.3 Transparency of Retrieval and Reasoning . . . 15
2.1.4 Discussion and Implications . . . 15
2.2 Claude Code . . . 16
2.2.1 Retrieval Strategies . . . 16
2.2.2 Accuracy and Latency Characteristics . . . 17
2.2.3 Transparency of Retrieval and Reasoning . . . 17
2.2.4 Discussion and Implications . . . 18
2.3 Pre-Project Meadow9+ . . . 19
2.3.1 Retrieval Strategies . . . 19
2.3.2 Accuracy and Latency Characteristics . . . 20
2.3.3 Transparency of Retrieval and Reasoning . . . 20
2.3.4 Discussion and Implications . . . 21

3 Proposed System Architecture **22**
3.1 System Overview . . . 22
3.2 Retriever System Architecture . . . 24
3.2.1 Initial Retrieval . . . 24
3.2.2 Rank Fusion and Reranking . . . 25
3.3 Evaluation System Architecture . . . 26
3.3.1 Dataset Preparation . . . 26
3.3.2 Automated Evaluation Pipeline . . . 26
3.3.3 Speed Benchmarking Setup . . . 27

4 Implementation and Progress to Date **28**
4.1 External Resources Incorporated . . . 28

4.1.1 Core Framework Libraries 28
4.1.2 Models Integration 28
4.1.3 Other Libraries 29
4.2 Retriever System Implementation 29
4.2.1 Transcript Chunking Implementation 29
4.2.2 Embedding Generation and Vector Storage 32
4.2.3 Multi-Stage Retrieval Process 35
4.2.4 Context Assembly and Prompt Construction 56
4.3 Stretch Goals and Deployment 62

5 Evaluation and Results **63**
5.1 Evaluation Pipeline Implementation 63
5.1.1 Dataset Generation Methodology 63
5.1.2 Retrieval Metrics 67
5.1.3 Generation Metrics 69
5.1.4 Separation of Retrieval and Generation Evaluation 71
5.1.5 Speed Evaluation Pipeline 72
5.1.6 Statistical Computation 73
5.2 Experimental Results 74
5.2.1 RAG Quality Metrics 74
5.2.2 Speed Performance 78

6 Conclusion and Future Work **80**
6.1 Future Work 80
6.1.1 Limited Coverage of Question Types in the Evaluation Pipeline . 80
6.1.2 RAG Pipeline Optimization for Ambiguous Queries 81
6.1.3 Production Deployment of Retrieval Server 81

Bibliography **85**

List of Figures

1.1 Gantt chart representing the project timeline 11

2.1 Otter.ai iOS application interface showing keyword search, AI chat, and summarization features 14

3.1 System integration: transcript indexing pipeline 23
3.2 Indexing pipeline: processing, chunking, and embedding stages 23
3.3 Query phase: retrieval and response generation 24
3.4 RAG pipeline: query-to-response architecture 24
3.5 Hybrid retrieval: multi-signal fusion and reranking 25
3.6 Dataset generation: QA pair creation from transcripts 26
3.7 Evaluation pipeline: retrieval and generation metrics 26
3.8 Benchmarking: RAG vs. pure LLM comparison 27

4.1 SRT input format with sequence numbers and timestamps 29
4.2 Parsed SRT structure with speaker metadata extraction 30
4.3 Token-based merging algorithm for chunk size optimization 31
4.4 LangChain Document object creation with chunk metadata 31
4.5 Document processing pipeline 33
4.6 Vector store creation from documents 34
4.7 Incremental document addition 34
4.8 ChromaDB storage directory structure 35
4.9 Vector search core function showing semantic retrieval with optional Chroma metadata filtering 36
4.10 Chroma metadata filter construction for transcript scoping 36
4.11 BM25 tokenization function using lowercase split 37
4.12 BM25 index initialization with corpus tokenization 38
4.13 BM25 query tokenization and metadata filtering 39
4.14 BM25 scoring and top-k ranking 39
4.15 BM25 result document construction 40
4.16 BM25 index persistence via pickle serialization 40
4.17 Phonetic index database schema with SQLite table and indices 41
4.18 Phonetic word tokenization and filtering 42
4.19 Double Metaphone phonetic encoding 43
4.20 Phonetic encoding examples for homophone matching 43
4.21 Phonetic index building with batch insertion 44
4.22 Phonetic retrieval SQL query with transcript filtering 45
4.23 Phonetic retrieval SQL query without filtering 45
4.24 Phonetic score conversion and document retrieval 46

4.25 Rare token detection using Zipf frequency 47
4.26 Extracting rare tokens from query 48
4.27 Fetching candidate document pool for fuzzy matching 48
4.28 Fuzzy scoring algorithm - setup and initialization 49
4.29 Fuzzy scoring algorithm - nested loop with RapidFuzz ratio 50
4.30 Fuzzy top-k selection and ranking 51
4.31 Reciprocal Rank Fusion weights for hybrid retrieval signals 53
4.32 Request payload preparation 54
4.33 Expected response format 55
4.34 Sorting by relevance score 55
4.35 Mapping results to original documents 56
4.36 Top-n selection 56
4.37 Chunk formatting with individual metadata headers 57
4.38 Output Format Example 57
4.39 Citation formatting for user transparency and source attribution 58
4.40 RAG system message template defining assistant role and constraints . . 59
4.41 Human message template with reasoning suppression directive 60
4.42 Complete RAG chat prompt construction with history support 60
4.43 Chat history structure for multi-turn conversation support 61
4.44 Streaming response handling for real-time token display 61

5.1 Context window grouping function signature with metadata tracking . . 64
5.2 Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata 64
5.3 LangChain pipe syntax for QA generation chain 65
5.4 LLM API call for QA pair generation with temperature 0.7 65
5.5 QA generation prompt template with requirements and format 66
5.6 Ground truth JSON format example with timestamps 67
5.7 Hit Rate formula for retrieval evaluation 67
5.8 Interval intersection algorithm for retrieval relevance checking 68
5.9 Mean Reciprocal Rank (MRR) formula and definition 68
5.10 F1 score calculation with precision and recall 69
5.11 Text preprocessing for token-level comparison 69
5.12 Precision, recall, and harmonic mean calculation with Counter intersection 70
5.13 LLM judge prompt template for semantic correctness evaluation 70
5.14 Retrieval failure scenario diagnostic example 71
5.15 Generation failure scenario with successful retrieval 72
5.16 Perfect performance scenario for both retrieval and generation 72
5.17 Interleaved execution pattern for unbiased speed benchmarking 73

5.18 Detailed success case example with perfect retrieval and generation . . . 75
5.19 Failure case example with retrieval success but generation error 76
5.20 False negative example with more specific generated answer 77

List of Tables

1.1 Detailed project schedule 12

4.1 Third-party libraries and frameworks categorized by functionality including LangChain, ChromaDB, and retrieval tools 29

4.2 Document metadata schema field descriptions 32

4.3 Search method characteristics comparing algorithm, indexing strategy, scoring approach, default retrieval limits, and primary use cases 52

5.1 Ground truth schema field descriptions 67

5.2 Statistical metrics table for latency analysis 73

5.3 Comparative metrics between RAG and Pure LLM approaches 74

5.4 Per-transcript RAG quality metrics aggregated results 74

5.5 Per-transcript speed comparison showing speedup factors 78

5.6 Quality-speed trade-off analysis across transcript sizes 79

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) represent a class of deep learning architectures trained on massive datasets, enabling them to comprehend and generate natural language with high versatility across domains [1]. As of 2025, approximately 67% of organizations worldwide have adopted LLMs to support their operational workflows [2]. One of the most prominent applications of LLMs is in conversational agents or chatbots [3]. These systems often integrate retrieval pipelines [4] that extract domain-specific information from a database or knowledge store, and provide the retrieved content as context to the LLM for response generation. This architecture allows users to query against proprietary or organizational datasets using natural language and receive accurate, context-specific responses.

Meadow9+, a speech recognition application designed to record and transcribe long-duration meetings, is one such platform that integrates an LLM-powered chatbot to enable retrieval of information from meeting transcripts. The platform is widely utilised by corporations, welfare organizations, and medical professionals who rely on accurate transcription of meetings. Since meetings are where critical discussions and decisions take place, these transcripts effectively serve as a knowledge base for the organization. The integrated chatbot therefore plays a crucial role in extracting actionable insights from this knowledge, and the system must ensure both accuracy and responsiveness of the retrieval process.

Currently, the Meadow9+ chatbot processes full transcripts directly as input to the LLM. This design leads to two major limitations. First, excessively long inputs exacerbate the "lost in the middle" phenomenon [5], where the LLM exhibits degraded performance when reasoning over information embedded in the middle sections of extended context windows. Second, longer context sequences increase computational overhead, thereby slowing inference and degrading user experience [6]. Furthermore, transcription outputs often contain inaccuracies and inconsistencies [7], and the current chatbot system lacks functionalities to handle noisy and ambiguous transcript data. These factors reduce usability and limit the effectiveness of the retrieval process for users who need quick and accurate access to meeting information.

1.2 Motivations and Objectives

Meeting transcripts only become truly valuable and actionable when users can quickly locate decisions, tasks, and explanations. However, Meadow9+ currently lacks an effective

retrieval mechanism to support such use cases, which significantly limits the practical utility of long-duration transcripts. As mentioned, the existing Meadow9+ chatbot employs a monolithic prompting approach, in which entire transcripts are provided directly to the LLM. This design leads to high latency, increased computational overhead, and degraded user experience, rendering the system unreliable in real-world scenarios. Improving information retrieval within Meadow9+ therefore directly addresses a critical gap in the current system and has immediate applicability in real-world deployments.

This project presents an opportunity to apply modern information retrieval (IR) and LLM integration techniques beyond naive prompting. In particular, Retrieval-Augmented Generation (RAG) combines classical retrieval methods with generative models, enabling systems to retrieve relevant document segments before generation [8]. Furthermore, the project also enables exploration of advanced retrieval enhancements, including query preprocessing, phonetic encoding, and fuzzy matching.

Motivated by these considerations, this project aims to design and implement a comprehensive transcript retrieval system that improves both retrieval accuracy and end-to-end system performance for long meeting transcripts. Specifically, the objectives are:

- Build a transcript retrieval pipeline that indexes long meeting transcripts into retrievable units and supports semantic retrieval via embeddings.
- Integrate retrieval into the chatbot using a RAG workflow to avoid full-transcript prompting, improving latency while maintaining answer quality.
- Develop a comprehensive evaluation pipeline to measure retrieval quality and end-to-end question answering performance using standard metrics, including hit-based metrics, reciprocal-rank metrics, and QA-style correctness metrics.
- Conduct empirical benchmarking comparing RAG against the existing pure LLM baseline under realistic conditions, with a focus on both accuracy and response latency.
- Explore extensible retrieval techniques tailored to noisy ASR transcripts, enabling efficient date-based and keyword-based retrieval in addition to chatbot-driven querying.

1.3 Scope

The scope of this project primarily focuses on the design and implementation of a RAG pipeline tailored for long-duration meeting transcripts. This includes developing appropriate transcript preprocessing strategies, such as chunking and embedding, to enable efficient and scalable retrieval from large textual documents. Particular emphasis is placed on handling the characteristics of long-form transcripts, where individual segments may vary in length and informational density, and ensuring that the resulting representations are suitable for downstream retrieval and generation tasks.

In addition to system design, the project places strong emphasis on quantitative evaluation and performance analysis. Retrieval effectiveness is systematically assessed using established information retrieval metrics, including Hit Rate, Mean Reciprocal Rank (MRR) [9], F1-score, and Accuracy, to provide an objective measure of retrieval quality. Furthermore, speed benchmarking is conducted to evaluate system efficiency, covering both end-to-end response latency and component-level performance across the retrieval and generation pipeline. The scope also includes exploratory work on system extensibility, laying the groundwork for integrating advanced retrieval mechanisms that can further enhance robustness and recall in noisy transcript environments.

Several aspects are explicitly considered out of scope to maintain a clear focus on retrieval and chatbot optimization. Improvements to the underlying Automatic Speech Recognition (ASR) model are excluded, as the project assumes transcripts are provided by the existing Meadow9+ transcription system. Similarly, user interface and user experience (UI/UX) redesign of the Meadow9+ application is not addressed. Finally, the project does not involve training or fine-tuning custom large language models, and instead focuses on optimizing system-level integration and retrieval strategies using pre-existing LLMs.

1.4 Contributions

Note that this section only provides an overview of the key contributions made. Additional details will be discussed in subsequent chapters, where they are grouped and presented differently. A summary of key contributions to this project is presented below:

Developed a hybrid transcript retrieval pipeline that indexes long-form meeting transcripts using token-based chunking and combines semantic, lexical search, phonetic search, and fuzzy matching. The pipeline employs Reciprocal Rank Fusion (RRF) [10] and cross-encoder reranking [11] to surface the top-5 most relevant chunks, achieving 96.7% Hit Rate and 0.941 Mean Reciprocal Rank across six diverse transcripts, demonstrating highly accurate retrieval without manual segmentation.

Integrated the retrieval pipeline into a RAG-based chatbot using LangChain orchestration, replacing full-transcript prompting with context-aware retrieval. This approach leverages per-transcript metadata filtering and temporal alignment, delivering an average 5.39× speedup over the pure LLM baseline, directly addressing latency and token cost constraints in production conversational interfaces.

Implemented a comprehensive evaluation framework measuring both retrieval quality (Hit Rate, MRR) and generation quality (F1 score, LLM-as-judge accuracy). The framework validates temporal overlap between retrieved chunks and ground-truth spans, providing multi-dimensional assessment of RAG system performance. Evaluation on 62 automatically generated QA pairs demonstrates 78.9% semantic correctness while maintaining sub-second retrieval latency (6% of total response time).

Conducted controlled empirical benchmarking comparing RAG against a pure LLM baseline under realistic conditions. Component-level timing measurements isolate retrieval overhead (mean: 0.42–0.55s) from generation time, and interleaved query ex-

ecution mitigates prompt caching bias. Results demonstrate that RAG achieves 78.9% accuracy at 5.39× speedup compared to baseline, establishing practical viability for real-time transcript question-answering in resource-constrained environments.

Explore extensible retrieval techniques tailored to noisy ASR transcripts, enabling efficient date-based and keyword-based retrieval in addition to chatbot-driven querying. This part is still a work in progress. The exact detail will be provided in the final report once everything has been finalized.

1.5 Project Schedule

The detailed project schedule is presented in Table 1.1, with the corresponding timeline visualization shown in Figure 1.1.

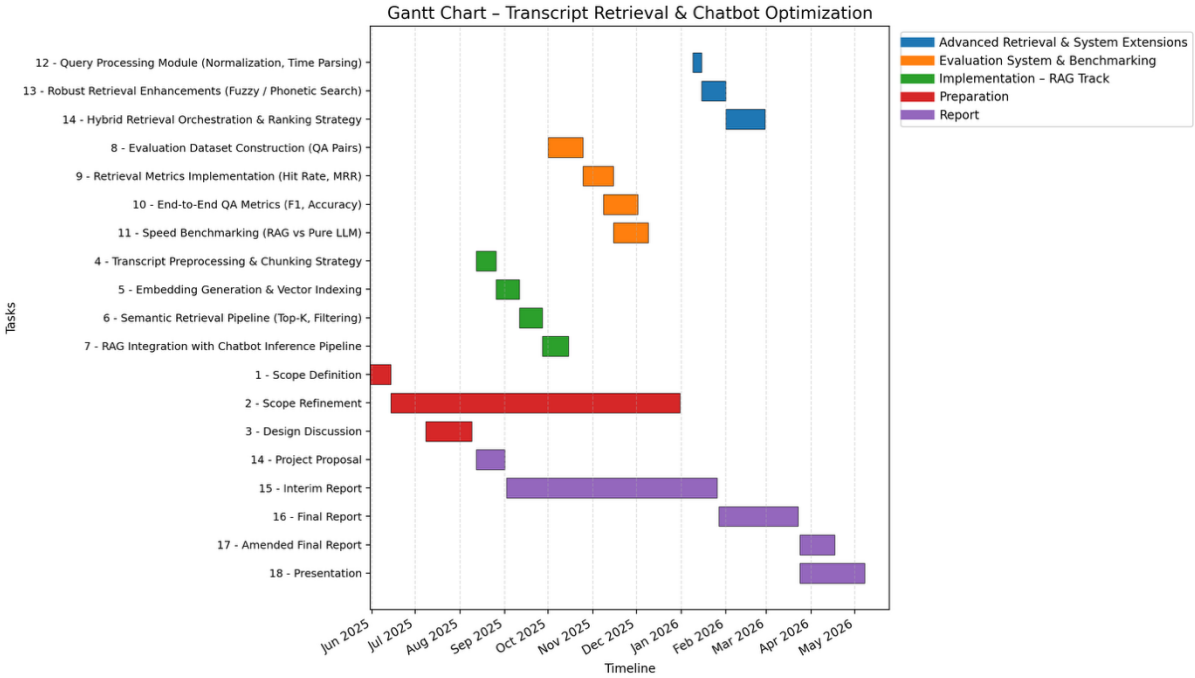

Figure 1.1: Gantt chart representing the project timeline

Index	Task Name	Start Date	End Date
Preparation			
1	Scope Definition	31 May 2025	14 Jun 2025
2	Scope Refinement	14 Jun 2025	31 Dec 2025
3	Design Discussion	8 Jul 2025	9 Aug 2025
Implementation – RAG Track			
4	Transcript Preprocessing & Chunking Strategy	12 Aug 2025	26 Aug 2025
5	Embedding Generation & Vector Indexing	26 Aug 2025	11 Sept 2025
6	Semantic Retrieval Pipeline (Top-K, Filtering)	11 Sept 2025	27 Sept 2025
7	RAG Integration with Chatbot Inference Pipeline	27 Sept 2025	15 Oct 2025
Evaluation System & Benchmarking			
8	Evaluation Dataset Construction (QA Pairs)	1 Oct 2025	25 Oct 2025
9	Retrieval Metrics Implementation (Hit Rate, MRR)	25 Oct 2025	15 Nov 2025
10	End-to-End QA Metrics (F1, Accuracy)	8 Nov 2025	2 Dec 2025
11	Speed Benchmarking (RAG vs Pure LLM)	15 Nov 2025	9 Dec 2025
Advanced Retrieval & System Extensions			
12	Query Processing Module (Normalization, Time Parsing)	9 Jan 2026	15 Jan 2026
13	Robust Retrieval Enhancements (Fuzzy / Phonetic Search)	15 Jan 2026	1 Feb 2026
14	Hybrid Retrieval Orchestration & Ranking Strategy	1 Feb 2026	28 Feb 2026
Report			
14	Project Proposal	12 Aug 2025	1 Sept 2025
15	Interim Report	2 Sept 2025	26 Jan 2026
16	Final Report	27 Jan 2026	23 Mar 2026
17	Amended Final Report	24 Mar 2026	17 Apr 2026
18	Presentation	24 Mar 2026	8 May 2026

Table 1.1: Detailed project schedule

Chapter 2

Literature Review

In today's competitive landscape, modern transcription-based applications and meeting asssistants increasingly integrate chat-based interfaces to allow users to query recorded conversations. In approaching the development of this project, a comprehensive review was conducted on the retrieval capabilities of three existing products: Otter.ai, Claude Code and the existing Meadow9+ mobile application. These products were selected due to their relevance to the proposed project.

The literature review focused on three key aspects: retrieval strategies, accuracy and latency characteristics, and transparency of retrieval and reasoning. Instead of trying to replicate existing solutions, this review aimed to understand how these platforms effectively address retrieval challenges and identify opportunities for improvement in developing our project. An in-depth analysis of the technological stacks and architectures will be presented in later chapters, specifically in the context of Meadow9+, as such information is not publicly available for the other products reviewed.

This chapter is divided into three main sections. Section 2.1 reviews Otter.ai, and Section 2.2 examines Claude Code, focusing exclusively on their retrieval system. Section 2.3 evaluates the Meadow9+ mobile application prior to this project.

2.1 Otter.ai

Otter.ai is a widely adopted speech-to-text and meeting assistant platform that provides real-time transcription, post-meeting summaries, and multiple mechanisms for retrieving information from recorded conversations [12]. As a mature commercial system, it represents a strong baseline for evaluating existing approaches to transcript retrieval and conversational interaction.

Otter.ai caters to a wide range of users, including businesses and students [13, 14]. While it is primarily used for transcribing meetings, the platform also proves valuable for sales calls and lecture notes [13, 14]. While Otter.ai offers applications on different platforms including web, iOS and Android [15], this discussion will focus exclusively on the iOS application.

2.1.1 Retrieval Strategies

Otter.ai provides three primary mechanisms for retrieving information from transcripts: keyword-based search, LLM-powered conversational querying (Otter AI Chat), and LLM-Based Summarization, as illustrated in Figure 2.1 [12, 15].

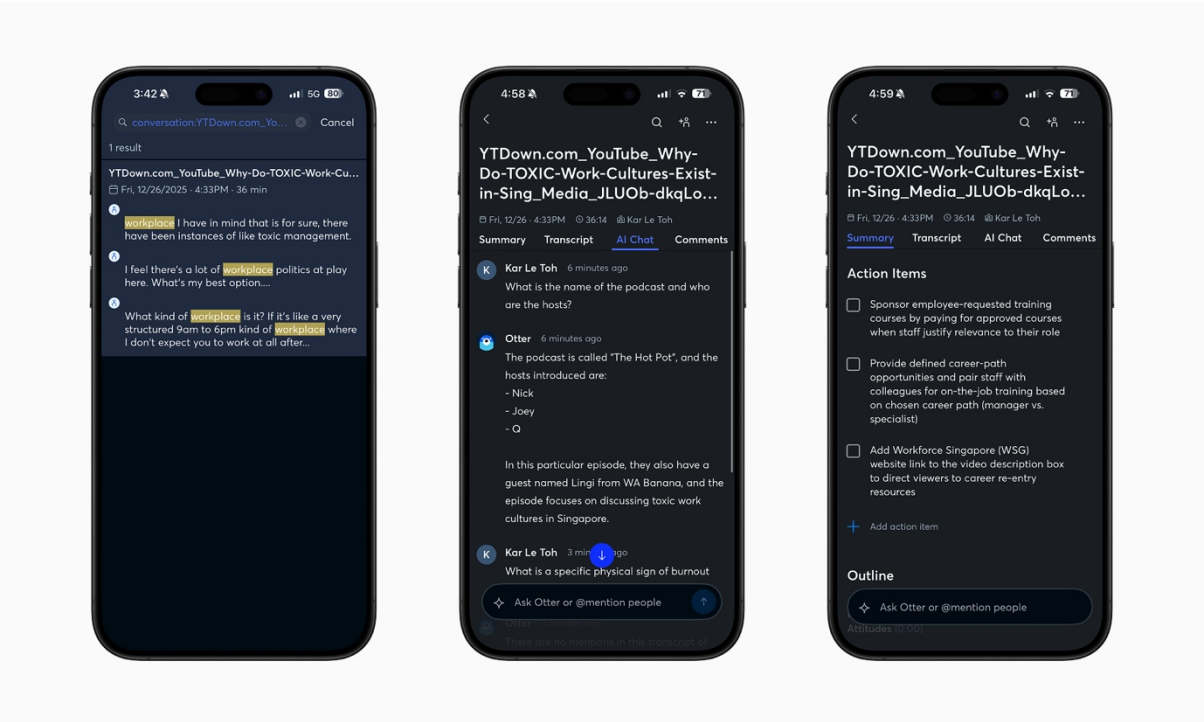

Figure 2.1: Otter.ai iOS application interface showing keyword search, AI chat, and summarization features

Keyword-Based Retrieval

Otter.ai supports keyword-based search over notes and transcripts [15]. Although Otter.ai does not publicly disclose the exact retrieval algorithm used, empirical testing suggests that the system employs a term-weighted ranking function such as TF-IDF or BM25. For example, queries containing additional discriminative terms such as "toxic workplace" versus "workplace" significantly alter result rankings in a manner consistent with BM25-style term frequency saturation and inverse document frequency weighting [16]. In addition, Otter.ai supports date-based filters over transcript metadata [15]. This feature substantially improves usability and demonstrates the value of combining structured metadata filtering with unstructured text search.

LLM-Based Conversational Retrieval (Otter AI Chat)

Otter AI Chat allows users to ask free-form natural language questions either across all transcripts or within a specific transcript. While Otter.ai does not explicitly disclose its backend architecture, its FAQ indicates that transcript content and prompts are transmitted to third-party services only when users actively interact with Otter AI Chat, strongly suggesting reliance on externally hosted LLMs from third-party providers [17].

Behavioral observations indicate that the global chat interface likely employs a retrieval step to narrow down relevant transcripts before passing context to the LLM, as responses often include links to specific conversations. This behavior is consistent with a RAG pipeline operating at the document level. In contrast, the per-transcript chat interface appears to process the entire transcript holistically, as responses rarely reference precise supporting segments or timestamps.

Notably, Otter AI Chat responses tend to be abstractive and conversational, favoring rephrased summaries over verbatim quotations. This design choice may be driven by prompt engineering rather than retrieval limitations; however, it also suggests limited emphasis on fine-grained, chunk-level semantic retrieval at query time.

LLM-Based Summarization

Beyond interactive retrieval, Otter.ai generates an extensive per-transcript summary using LLM-based processing [12]. This summary is structured into three layers: (i) a high-level overview of the meeting, (ii) extracted action items, and (iii) a hierarchical outline organized by timestamps, with further speaker-segmented summaries nested within each time interval. This multi-level summarization demonstrates effective use of LLMs to transform long-form transcripts into structured, navigable representations.

2.1.2 Accuracy and Latency Characteristics

From manual evaluation, Otter.ai's keyword-based search exhibits both high accuracy and negligible latency, consistent with expectations for traditional lexical retrieval systems. This makes keyword search the most reliable and responsive mechanism for precise information lookup. Otter AI Chat, when applied to transcripts up to approximately two hours in length, also demonstrates low perceived latency and strong answer accuracy. This performance is likely enabled by the use of large, cloud-hosted LLMs with sufficient context windows and optimized inference infrastructure.

2.1.3 Transparency of Retrieval and Reasoning

A notable limitation of Otter AI Chat is its lack of transparency. The system provides minimal visibility into which transcript segments support a given answer, and users cannot inspect or verify the retrieved evidence. This opacity limits user trust and makes error diagnosis difficult, particularly in professional or compliance-sensitive settings.

2.1.4 Discussion and Implications

Overall, Otter.ai exemplifies a feature-rich and well-integrated retrieval ecosystem for meeting transcripts. It combines global and local keyword search, conversational querying, and comprehensive summarization to support diverse user workflows. These capabilities collectively demonstrate that effective transcript systems must treat retrieval as a first-class design concern, rather than a secondary feature layered on top of transcription.

However, the apparent lack of fine-grained, chunk-level grounding and the absence of explicit evidence attribution indicate that Otter.ai's system prioritizes fluency and abstraction over verifiability. While this design choice may be acceptable for general meeting summarization, it introduces risks of hallucination and reduces user trust in scenarios where factual accuracy and traceability are critical. This limitation is particularly rele-

vant to Meadow9+, since it cannot rely on large cloud-hosted frontier models with extensive context windows and optimized inference pipelines. Instead, it employs a smaller, self-hosted LLM with stricter latency and context constraints. Consequently, passing full transcripts, or even large document-level contexts, into the model is neither scalable nor reliable.

These observations collectively imply that Meadow9+ must adopt a *more retrieval-centric and evidence-grounded design* than Otter.ai. Rather than treating retrieval as a preliminary filtering step, retrieval must be a first-class system component that operates at multiple granularities (document-level, chunk-level, and temporal level). Figure 2.1 illustrates the baseline chatbot architecture commonly observed in existing systems, where retrieval is shallow or opaque and tightly coupled to generation.

In contrast, Meadow9+ is designed to decouple retrieval from generation and to emphasize transparent, inspectable retrieval results. This includes (i) hybrid lexical–semantic retrieval to handle both exact terms and paraphrased queries, (ii) phonetic and fuzzy matching to mitigate transcription errors, and (iii) chunk-level evidence selection to reduce prompt length. By explicitly surfacing retrieved segments and their timestamps, Meadow9+ aims to improve both answer accuracy and user trust, which is an aspect largely absent in Otter.ai's conversational interface.

2.2 Claude Code

Recent advances in LLM–based agents have extended beyond conversational assistants into domain-specialized systems that tightly integrate retrieval, reasoning, and action. One notable example is Claude Code, a command-line interface (CLI) agent developed by Anthropic that brings LLM-driven reasoning directly into the software development workflow [18]. Claude Code is widely regarded as one of the most capable coding agents currently available, not solely due to model quality, but because of its system-level retrieval and context management design that enables effective reasoning over large, evolving codebases.

2.2.1 Retrieval Strategies

Claude Code adopts an agentic retrieval paradigm, where the LLM actively searches and retrieves relevant context from the environment at query time rather than relying on a precomputed embedding index or static vector store. Specifically, Claude Code is granted controlled read access to the local project directory and retrieves information by invoking standard developer tools such as file search and content matching when needed [19, 20]. Retrieved files or code snippets are then injected directly into the model's context window.

Importantly, retrieval is performed dynamically through tool calls that operate over the source-of-truth codebase rather than through a precomputed embedding index [19, 20]. This design avoids issues associated with stale embeddings, incomplete indexing, and semantic drift, ensuring that the retrieved context always reflects the most recent state

of the project.

Claude Code also supports persistent project-level memory via a lightweight mechanism: a configuration file (e.g., CLAUDE.md) that stores high-level instructions, architectural conventions, and recurring commands. This file is automatically injected into every session, acting as a form of long-term context without requiring a separate memory model or database [21].

2.2.2 Accuracy and Latency Characteristics

The agentic retrieval strategy in Claude Code introduces clear trade-offs between accuracy and latency. On the one hand, dynamically searching the codebase yields high retrieval precision, as the agent can iteratively refine its search queries and inspect exact source files relevant to the task [19]. This approach can improve task success rates in complex, multi-file coding scenarios compared to static retrieval pipelines.

On the other hand, retrieval incurs non-trivial overhead. Each file read and search operation increases prompt size and introduces runtime latency due to tool execution and additional token processing. This design typically leads to higher token usage and slower individual responses compared to systems that rely on pre-indexed embeddings, but the overhead can be acceptable given the resulting improvement in task accuracy and robustness.

To further improve accuracy on complex tasks, Claude supports an extended thinking mode in which the model spends more time deliberating before producing an answer [22]. While this increases latency, it can improve solution quality for tasks requiring multi-step reasoning or global code understanding.

2.2.3 Transparency of Retrieval and Reasoning

A defining characteristic of Claude Code is its emphasis on transparent retrieval and execution. Tool use is permission-gated, and any action that may modify the system requires user approval [19, 20]. This design allows developers to observe how context is retrieved and to intervene when necessary.

Claude Code also provides a "Plan Mode," in which the agent analyzes the codebase and proposes a sequence of actions without executing them. This separation of planning and execution improves trust and enables users to validate the agent's understanding before committing changes [18].

From a system-design perspective, Claude Code demonstrates that retrieval transparency is not only a safety feature but also a debugging aid, allowing users to reason about failures in retrieval or context selection.

2.2.4 Discussion and Implications

Although Claude Code operates in the domain of software engineering, its retrieval design offers several transferable insights for transcript-based systems such as Meadow9+.

First, Claude Code's avoidance of static embedding-based retrieval highlights a broader design principle: retrieval systems must align with the nature of the underlying data. Claude Code adopts a fully agentic retrieval approach precisely because of the characteristics of its target domain. Dynamic, query-time search over the source-of-truth codebase ensures that retrieved context remains up-to-date and semantically precise, avoiding issues associated with stale or incomplete indices.

In contrast, Meadow9+ operates over transcript data that is static once transcription is completed. While a full agentic retrieval paradigm is powerful, it is not strictly required in this setting. Additionally, practical system constraints must also be considered, as Meadow9+ employs a smaller self-hosted LLM with substantially weaker reasoning and tool-use capabilities compared to commercial frontier models used in systems such as Claude Code. Under these constraints, fully agentic retrieval may introduce performance bottlenecks, increased latency, or even unstable retrieval behavior, particularly when iterative reasoning and tool invocation are required.

These observations suggest that the optimal retrieval strategy for Meadow9+ lies in a hybrid approach, combining embedding-based semantic retrieval with symbolic techniques. Such a design leverages the static nature of transcript data to enable efficient indexing while remaining robust to transcription noise and lexical variation. At the same time, it avoids the computational and reasoning overheads associated with fully agentic retrieval, making it more suitable for deployment with resource-constrained, self-hosted LLMs.

Second, Claude Code's trade-off between accuracy and latency offers guidance for Meadow9+'s system design. Claude Code deliberately trades off latency and token efficiency in exchange for higher task accuracy. Each retrieval step incurs additional overhead due to tool invocation, file inspection, and prompt expansion. This trade-off is acceptable in developer workflows, where correctness is often prioritized over response time and where commercial frontier models with strong reasoning capabilities are available.

Meadow9+, however, operates under stricter latency and cost constraints. The system also relies on a self-hosted, smaller-scale LLM, which is significantly less capable than proprietary frontier models. Under such constraints, a fully agentic retrieval loop may introduce excessive latency and unstable performance. Moreover, iterative retrieval decisions place additional reasoning burden on the model itself, increasing the risk of suboptimal search strategies and error propagation.

These observations suggest that the retrieval strategy for Meadow9+ should be front-loaded into deterministic or weakly learned components, such as BM25 and dense embeddings, rather than delegated to the LLM. This reduces the reasoning load on the model and yields more predictable latency characteristics.

Finally, Claude Code's emphasis on transparent retrieval suggests an additional opportunity for Meadow9+. While Meadow9+ targets non-technical users, partial transparency,

such as highlighting which transcript segments were used to generate an answer, can improve user trust and perceived reliability. This is especially important in professional meeting contexts where incorrect attribution or hallucinated decisions can have serious consequences.

Incorporating lightweight transparency mechanisms aligns with Claude Code's philosophy that retrieval visibility is not merely a safety feature but a debugging and validation aid. Such features can also support iterative refinement of retrieval strategies during system evaluation.

2.3 Pre-Project Meadow9+

Meadow9+ is a mobile speech-to-text application developed by researchers from the NTU Speech and Language Laboratory, with initial development dating back to approximately 2021 (exact date to be confirmed) [23,24]. It is a product of ABAX AI, a company specializing in advanced speech recognition technologies tailored for enterprise and professional use cases [24, 25].

The platform is primarily designed for Singapore-based business professionals, with a strong emphasis on one-to-one meetings as its core usage scenario [23–25]. Nevertheless, Meadow9+ also supports a broader range of transcription contexts, including multi-party discussions and news audio recordings. A key strength of the system lies in its ability to handle Singaporean accents and domain-specific terminology, addressing a notable gap in the local speech recognition market where generic transcription systems often underperform.

While Meadow9+ demonstrates strong transcription quality, particularly in localized linguistic contexts, its post-transcription information retrieval and interaction capabilities remain limited. The following subsections analyze the retrieval and summarization mechanisms implemented in the pre-project version of Meadow9+, highlighting their strengths and limitations.

2.3.1 Retrieval Strategies

Meadow9+ provides two primary mechanisms for retrieving information from transcripts: (1) text-matching search and (2) LLM-based conversational querying.

Text-Matching Search

The system supports a text-matching search functionality at the global level, allowing users to search across conversation titles. At query time, the search matches user-provided keywords against stored conversation titles, returning transcripts whose metadata contains the queried terms. This design emphasizes simplicity and responsiveness, making it suitable for lightweight retrieval scenarios where meaningful titles are consistently available.

LLM-Based Conversational Retrieval

Meadow9+ integrates an LLM-powered chatbot that allows users to pose free-form natural language questions within a selected transcript [26]. For privacy and data governance considerations, the chatbot relies on a self-hosted LLM.

In the pre-project system, the chatbot passes the full transcript directly into the LLM as input context at query time, without an intermediate retrieval or context selection stage. Responses generated by the system are primarily abstractive and conversational in nature, focusing on synthesized summaries rather than direct quotations or fine-grained references to specific transcript segments.

LLM-Based Summarization

In addition to interactive querying, Meadow9+ generates a per-transcript summary using LLM-based processing [26]. This summary provides a condensed overview of the meeting content and is generated independently of user queries. The summarization output is static and intended for post-meeting review rather than targeted information retrieval. It does not support interaction, filtering, or alignment with subsequent user questions.

2.3.2 Accuracy and Latency Characteristics

Based on manual evaluation, Meadow9+'s text-matching search demonstrates high accuracy and minimal latency under ideal conditions. When relevant metadata is present, search results are returned promptly.

In contrast, the chatbot exhibits high perceived latency and weak answer accuracy when applied to transcripts exceeding approximately 1 hour in length. These performance issues are likely attributable to two factors: (1) the use of a relatively small self-hosted LLM with limited reasoning capacity compared to commercial frontier models, and (2) inefficient utilization of the model's context window due to the absence of retrieval or context selection mechanisms.

2.3.3 Transparency of Retrieval and Reasoning

The pre-project Meadow9+ chatbot presents users with a single generated response per query. The system does not provide explicit indicators of which transcript segments were most relevant to the answer, nor does it surface supporting excerpts or references from the transcript. As a result, users interact with the chatbot as a black-box conversational interface, without visibility into intermediate reasoning or evidence selection processes.

2.3.4 Discussion and Implications

The pre-project Meadow9+ system demonstrates a clear separation between transcription quality and post-transcription interaction capabilities. While the platform is optimized for accurate speech recognition in localized contexts, its retrieval and interaction mechanisms are comparatively lightweight and monolithic, exhibiting several structural limitations that impact effectiveness, scalability, and user experience.

For text-matching search, the reliance on transcript-level metadata significantly constrains retrieval coverage. Because conversation titles and descriptions are optional, many transcripts lack sufficiently descriptive metadata, reducing the practical utility of this search mechanism. Moreover, the absence of keyword-based searching within transcript content prevents users from retrieving specific terms, entities, or discussion points embedded in the conversation itself. The system also does not support natural language or temporal constraints (e.g., "last Friday" or "earlier in the meeting"), which are increasingly expected in modern retrieval interfaces.

The LLM-based conversational retrieval approach addresses some expressiveness limitations by allowing free-form queries; however, its design introduces different trade-offs. Passing entire transcripts directly to the LLM leads to high inference latency for long-duration meetings and exacerbates the lost-in-the-middle problem. Additionally, the abstractive nature of responses, combined with the lack of explicit evidence grounding, reduces answer verifiability and makes systematic error analysis difficult.

Collectively, these observations highlight a fundamental architectural gap in the pre-project Meadow9+ system: the absence of an explicit, fine-grained retrieval layer that mediates between raw transcripts and language model generation. This gap motivates the adoption of a structured RAG framework in the proposed system. By decoupling retrieval from generation and incorporating hybrid retrieval strategies tailored to noisy speech-to-text data, the redesigned system aims to improve latency, accuracy, transparency, and scalability while preserving conversational usability.

Chapter 3

Proposed System Architecture

3.1 System Overview

The proposed system introduces a RAG–based architecture to address the scalability and latency limitations observed in the existing Meadow9+ chatbot, which directly supplies entire meeting transcripts to a self-hosted LLM. The redesigned architecture decomposes the end-to-end pipeline into modular components for transcript processing, retrieval, and generation, enabling efficient querying over long-duration and noisy speech-to-text data. [4]

To minimise disruption to the existing Meadow9+ infrastructure, the new retrieval system is deployed as a separate retrieval server that interacts with the current Meadow9+ backend server. This separation reduces the extent of modifications required on the production backend, while enabling the possibility of controlled A/B testing in production to compare the performance of the existing chatbot approach against the new retrieval pipeline proposed in this project.

In the redesigned workflow, when a user submits a new recording and a corresponding transcription is generated, the transcript continues to be stored in the existing database as per the current implementation. In addition, the same transcript is forwarded to the retrieval server, where it is parsed, embedded, and indexed for retrieval, as illustrated in Figure 3.1. This ensures that the proposed retrieval system can be integrated incrementally alongside the existing system, while maintaining backward compatibility and enabling direct evaluation under real usage conditions.

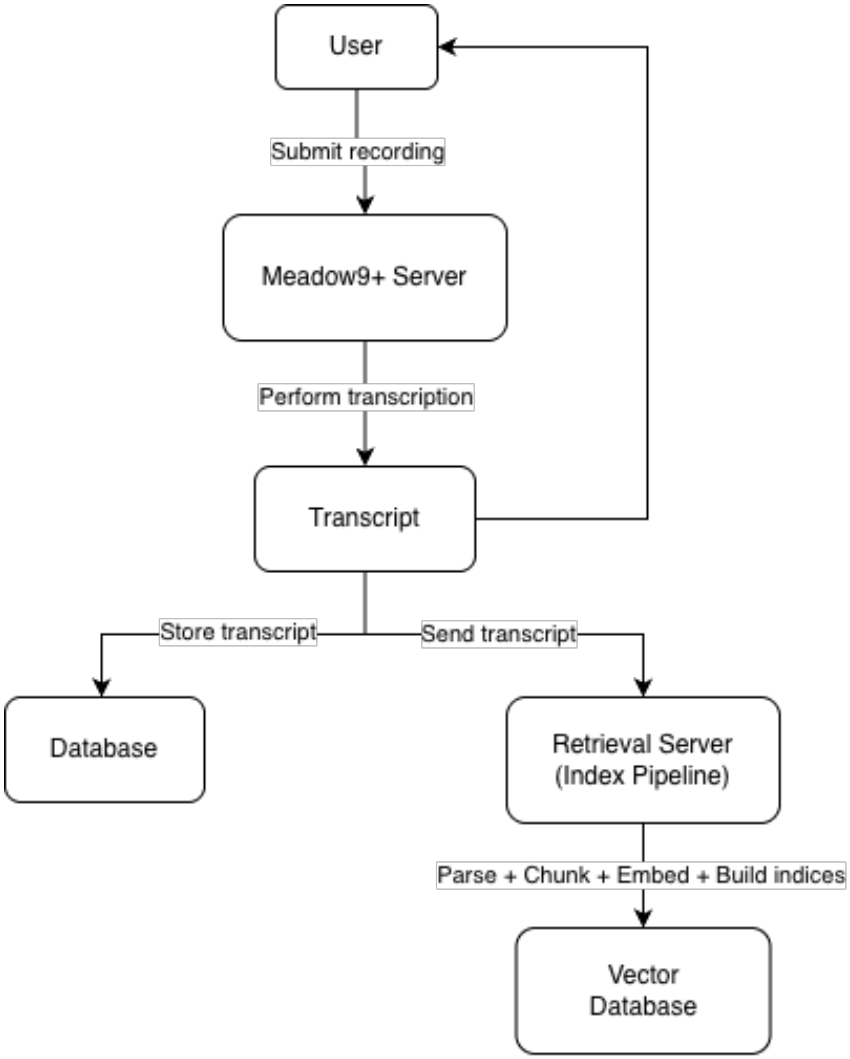

Figure 3.1: System integration: transcript indexing pipeline

At a high level, the retrieval system operates in two distinct phases: an indexing phase and an query phase.

During indexing, raw speech-to-text transcripts in SubRip (SRT) format are parsed, segmented, embedded, and stored in a persistent vector database alongside auxiliary keyword-based indices. Figure 3.2 shows the complete indexing pipeline, including processing, chunking, and embedding stages.

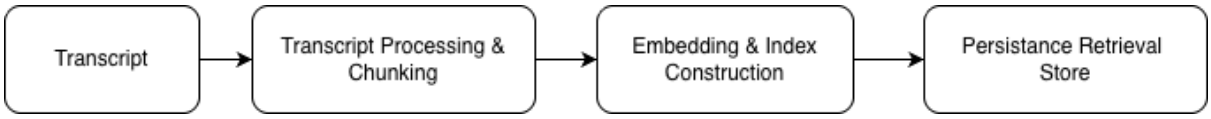

Figure 3.2: Indexing pipeline: processing, chunking, and embedding stages

During querying, user questions are transformed into embeddings, used to retrieve a small set of relevant transcript segments, and supplied to the LLM as contextual grounding for response generation. This query workflow is depicted in Figure 3.3, with the complete RAG pipeline architecture shown in Figure 3.4. [4]

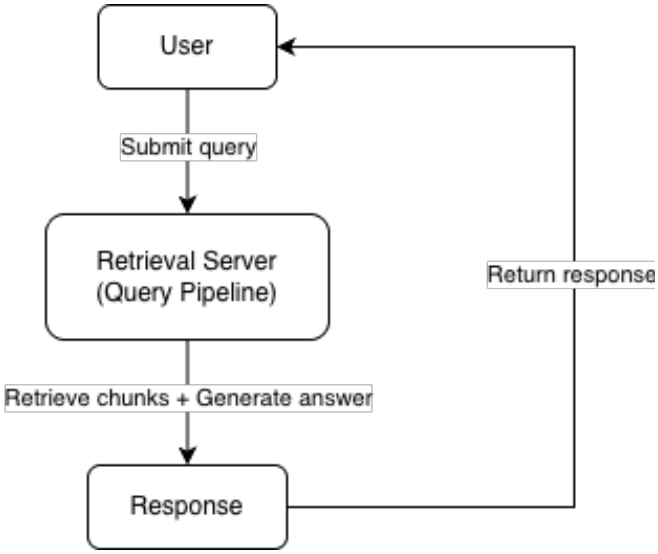

Figure 3.3: Query phase: retrieval and response generation

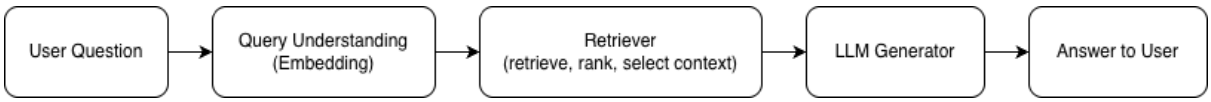

Figure 3.4: RAG pipeline: query-to-response architecture

3.2 Retriever System Architecture

3.2.1 Initial Retrieval

At query time, the user question is first embedded using the same embedding model as the indexed chunks. As shown in Figure 3.5, four retrieval strategies are then executed in parallel: semantic similarity search over the vector database, keyword-based retrieval using BM25, phonetic search, and fuzzy matching. [16, 27, 28] Each retriever returns a ranked list of candidate transcript chunks.

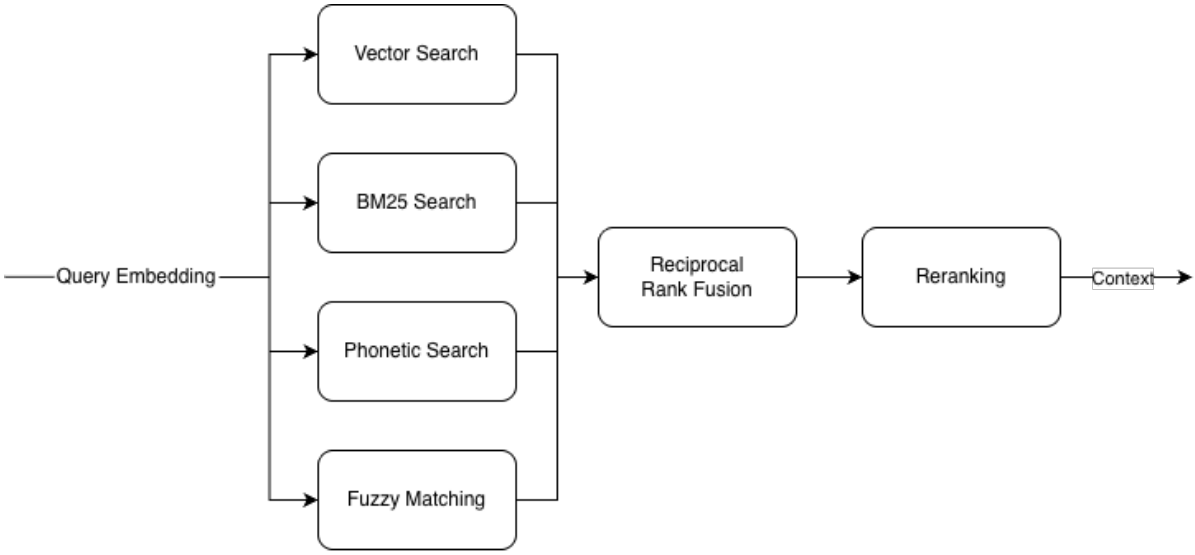

Figure 3.5: Hybrid retrieval: multi-signal fusion and reranking

This hybrid design is motivated by the nature of speech-to-text transcripts, which frequently contain recognition errors, disfluencies, and domain-specific terminology. [7] Semantic retrieval captures paraphrased or conceptually related content, while BM25 remains effective for exact keyword matches and rare terms. [16] Phonetic search improves recall when proper nouns or technical phrases are mis-transcribed into similar-sounding variants, and fuzzy matching further mitigates minor spelling inconsistencies and token-level noise. [27, 28] Together, these complementary signals increase retrieval robustness under noisy transcription conditions.

3.2.2 Rank Fusion and Reranking

The outputs of the semantic, lexical, and phonetic retrieval components are combined using Reciprocal Rank Fusion (RRF), which normalises rankings across heterogeneous retrieval functions and prioritises candidates that are consistently ranked highly across multiple retrievers. [10] RRF was selected due to its simplicity, robustness, and strong empirical performance in information retrieval systems.

A cross-encoder reranker is then applied to the fused candidate set. [11] The reranker jointly encodes the query and each candidate chunk to compute fine-grained relevance scores, improving precision at the top ranks. To ensure system robustness, the architecture is designed to degrade gracefully: if the reranker service is unavailable, the system falls back to the fused ranking without interrupting query processing.

3.3 Evaluation System Architecture

3.3.1 Dataset Preparation

To enable systematic evaluation, a dedicated pipeline generates question–answer pairs from transcript data using an external LLM, as illustrated in Figure 3.6. Transcript segments are grouped into medium-sized context windows, from which multiple self-contained questions are generated. Each question is annotated with the corresponding timestamp range, forming a temporally grounded ground truth dataset.

Figure 3.6: Dataset generation: QA pair creation from transcripts

3.3.2 Automated Evaluation Pipeline

Figure 3.7 shows the evaluation architecture, which mirrors the query pipeline (RAG system) but operates in batch mode. For each question, the system performs retrieval, generates an answer, and computes retrieval and generation metrics. Retrieval effectiveness is assessed using hit rate and mean reciprocal rank (MRR) based on timestamp overlap, [9] while generation quality is measured using token-level F1 score and an LLM-based semantic judge. [29]

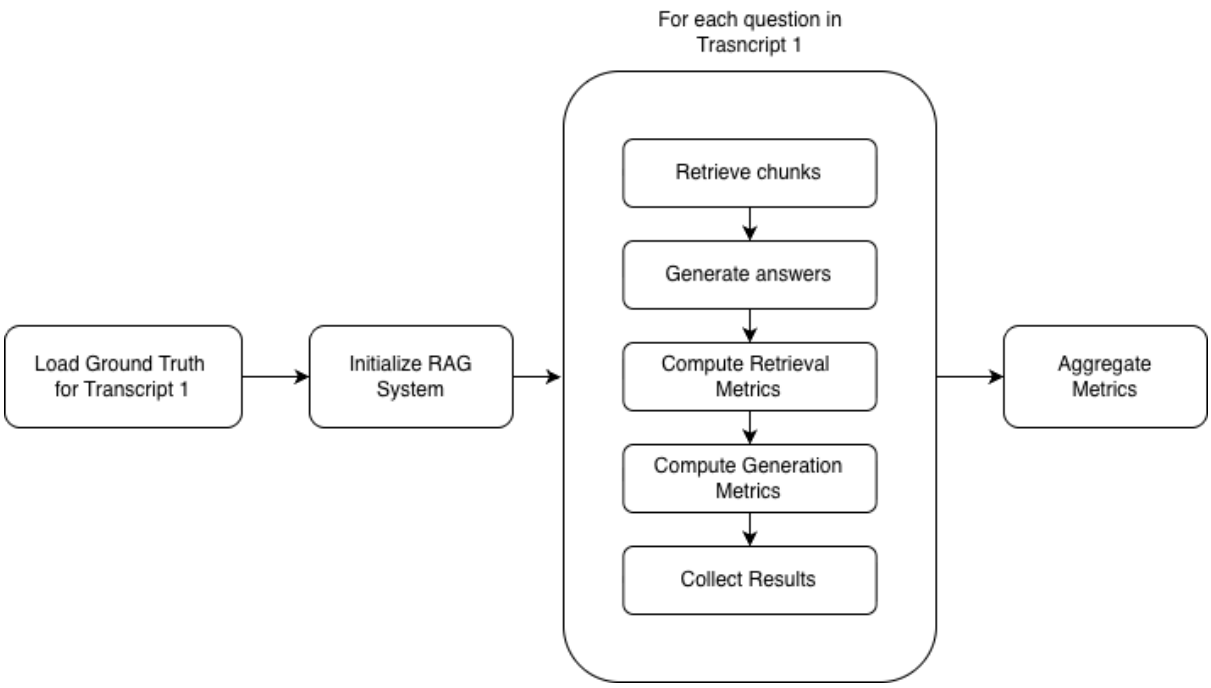

Figure 3.7: Evaluation pipeline: retrieval and generation metrics

Crucially, retrieval metrics are computed before generation, ensuring that retrieval and generation errors can be diagnosed independently.

3.3.3 Speed Benchmarking Setup

A separate benchmarking pipeline compares the RAG-based system against a pure LLM baseline that consumes truncated full transcripts, as shown in Figure 3.8. Queries are executed in an interleaved manner to mitigate caching effects, and component-level timings are recorded to isolate retrieval and generation costs. This design enables fair and reproducible latency comparisons across different transcript lengths.

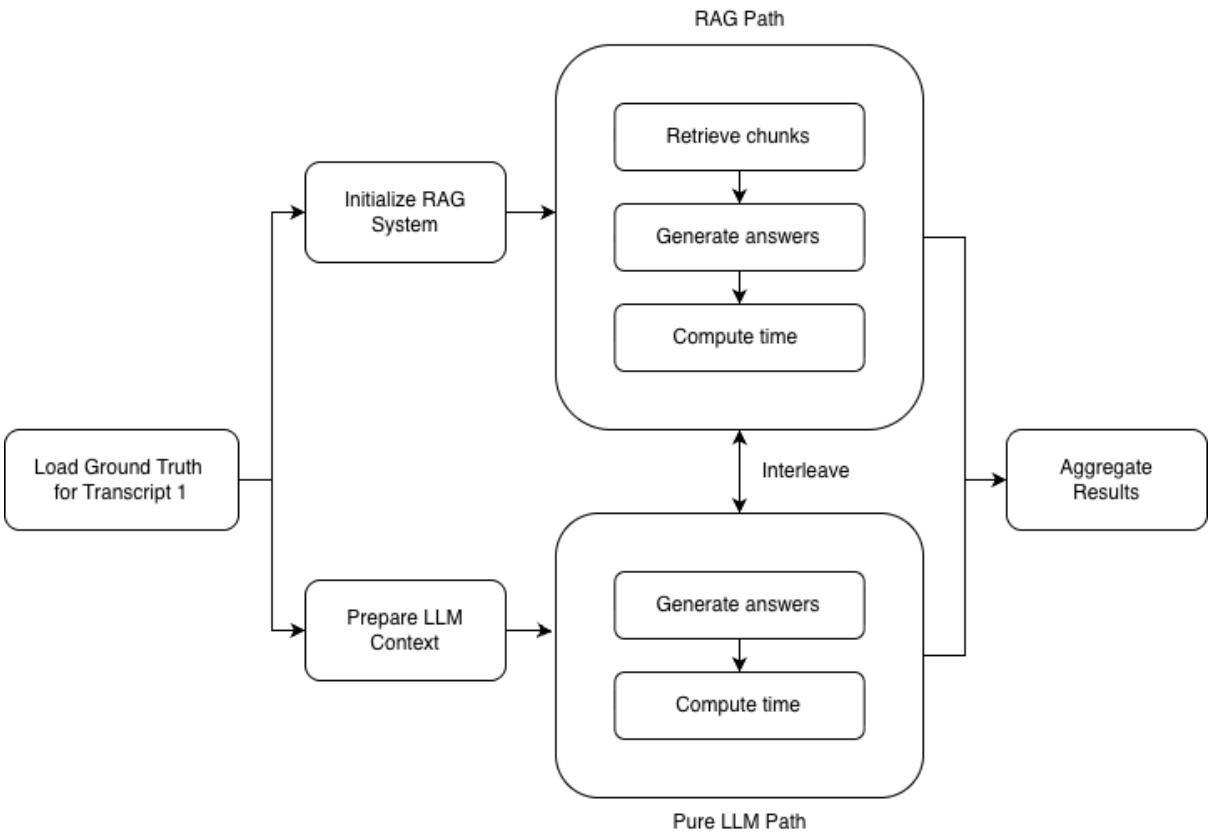

Figure 3.8: Benchmarking: RAG vs. pure LLM comparison

Chapter 4

Implementation and Progress to Date

Note that this chapter is still a work in progress. The exact description will be provided in the final report once everything has been finalized. The summary paragraphs below are provided as a draft.

This chapter presents the implementation details of the RAG system developed for the Meadow9+ transcription platform. It is organized into sections covering external resources and libraries, the retriever system implementation, and stretch goals and deployment.

4.1 External Resources Incorporated

The RAG system integrates multiple external libraries and services to provide comprehensive RAG capabilities, as summarized in Table 4.1. These resources are organized into six primary categories, each serving distinct functional requirements within the pipeline.

4.1.1 Core Framework Libraries

The system is built on the LangChain ecosystem, which provides core abstractions for document handling, prompt template management, and output parsing. The web API layer utilizes FastAPI as the REST framework with Pydantic for data validation and serialization, enabling type-safe request/response handling through structured schemas defined in the data model definitions. [30–34]

4.1.2 Models Integration

In the main RAG pipeline, a self-hosted jina-embeddings-v3-f16 model is used for document embedding, bge-reranker-v2-m3-FP16 is used for reranking retrieved candidates, and Qwen3-14B-Q4 serves as the answer generation LLM. For evaluation pipelines, DeepSeek, via the DeepSeek API, is used to generate ground-truth answers. All local models are hosted using llama.cpp server (llama-server), which provides efficient inference with optimized CPU/GPU acceleration and OpenAI-compatible API endpoints. [35–39]

4.1.3 Other Libraries

Other supporting libraries are consolidated in Table 4.1, which groups tools by their primary roles in retrieval, model integration, data processing, and development workflows. [40–52]

Category	Key Libraries	Primary Purpose
Framework	LangChain, FastAPI, Pydantic	RAG orchestration and REST API
Vector Store	Chroma	Persistent embedding storage and similarity search
Retrieval	rank-bm25, metaphone, rapidfuzz, wordfreq	Hybrid search with phonetic and fuzzy matching
LLM Integration	langchain-openai	OpenAI-compatible API wrapper for local/cloud models
Data Processing	dateparser, pandas, numpy, matplotlib	Natural language parsing and data analysis
Development	pytest, requests	Testing and external service communication

Table 4.1: Third-party libraries and frameworks categorized by functionality including LangChain, ChromaDB, and retrieval tools

4.2 Retriever System Implementation

4.2.1 Transcript Chunking Implementation

4.2.1.1 SRT Parsing

The SRT parser implementation converts SRT format subtitle files into structured Python dictionaries. Figure 4.1 shows the input SRT format, which consists of sequence numbers, timestamps, and text content:

```
1
00:00:00,640 --> 00:00:15,140
Hi, how are you today?

2
00:00:15,140 --> 00:01:31,240
>> Speaker: Read this aloud for me...
```

Figure 4.1: SRT input format with sequence numbers and timestamps

The parser produces structured output, as shown in Figure 4.2:

```
{
    'sequence': 1,
    'start_time': '00:00:00,640',
    'end_time': '00:00:15,140',
    'text': 'Hi, how are you today?',
    'speaker': None  # or 'Speaker' if ">>" prefix detected
}
```

Figure 4.2: Parsed SRT structure with speaker metadata extraction

Speaker detection handles two patterns: simple » prefix and » SpeakerName: format. The speaker name is extracted and removed from the text content, preserving speaker metadata separately.

4.2.1.2 Token-Based Merging

Token-Based Chunking Strategy

The chunking strategy employs actual tokenizer-based token counting rather than character-based estimation. This approach ensures accurate management of the LLM context window under strict input constraints, while maintaining consistency with the embedding model's token limits. By enforcing explicit token-based boundaries during chunk construction and retrieval, it prevents both underutilization of available chunk capacity and overflow beyond model limits. As a result, the system retains precise and predictable control over the total amount of retrieved context supplied to downstream generation components.

Preservation of Natural SRT Boundaries

Rather than applying arbitrary text splitting, the system preserves the original SRT subtitle boundaries during chunking to retain transcript structure and metadata fidelity. This design maintains speaker diarization integrity across chunks and preserves the natural conversational flow and turn-taking structure, which improves coherence during downstream retrieval and generation. In addition, it ensures timestamp metadata remains accurately aligned with the source transcript, enabling reliable temporal referencing during evaluation and user-facing responses. By respecting these semantic units, the subsequent merging strategy can combine segments more meaningfully without disrupting the underlying conversational context.

Small SRT chunks are merged to create appropriately sized segments for embedding and retrieval. Figure 4.3 shows how the merging algorithm operates:

```
1. For each chunk:
   a. Count tokens using TokenCounter
   b. If chunk < MIN_CHUNK_SIZE (25 tokens):
      - Add to accumulator
      - If accumulator >= TARGET_MERGED_SIZE (128 tokens):
        * Create merged chunk
        * Update timestamps (earliest start, latest end)
        * Track merged_count
   c. Else:
      - Flush accumulator if non-empty
      - Emit chunk as-is

2. Flush remaining accumulator
```

Figure 4.3: Token-based merging algorithm for chunk size optimization

4.2.1.3 Document Creation and Metadata Schema

The final stage converts parsed chunks into LangChain Document objects, as shown in Figure 4.4: [30]

```
Document(
    page_content=chunk['text'],
    metadata={
        'chunk_id': idx,
        'source': filename,
        'transcript_id': clean_filename,  # for filtering
        'total_chunks': len(chunks),
        'start_time': chunk['start_time'],
        'end_time': chunk['end_time'],
        'sequence': chunk['sequence'],
        'speaker': chunk.get('speaker'),
        'chunk_size_tokens': token_count,
        'merged_count': chunk.get('merged_count', 1),
        'overlap_tokens': chunk.get('overlap_tokens', 0)
    }
)
```

Figure 4.4: LangChain Document object creation with chunk metadata

Table 4.2 shows the document metadata schema field descriptions:

Field	Type	Description
chunk_id	int	Sequential ID within transcript (0-indexed)
source	str	Original filename (e.g., "example_transcript_1.srt")
transcript_id	str	Cleaned filename for metadata filtering
total_chunks	int	Total number of chunks in source transcript
start_time	str	Start timestamp in HH:MM:SS,mmm format
end_time	str	End timestamp in HH:MM:SS,mmm format
sequence	int	Original SRT sequence number
speaker	str/None	Detected speaker name from SRT markers
chunk_size_tokens	int	Token count via TokenCounter
merged_count	int	Number of original chunks merged (≥ 1)
overlap_tokens	int	Tokens overlapping with previous chunk

Table 4.2: Document metadata schema field descriptions

4.2.2 Embedding Generation and Vector Storage

4.2.2.1 Global Unified Index Architecture

All transcript chunks are stored in a single, global vector store rather than maintaining separate indices per transcript. This architectural decision provides several key advantages: simplified index management (one collection instead of dozens), reduced memory overhead (shared vocabulary and model weights), support for cross-transcript semantic search when queries span multiple documents, and unified metadata-based filtering for transcript scoping. Metadata-based filtering enables efficient restriction of search results to specific transcripts without requiring separate indices, as demonstrated in Figure 4.10 (shown in Section 4.2.3.1).

4.2.2.2 Embedding Service

The embedding functionality is implemented as a separate microservice, hosted via llama.cpp server. This separation from the main LLM service and reranker service enables independent scaling, resource allocation, and model versioning for each component.

The embedding service communicates via HTTP POST requests to the /v1/embeddings endpoint, following the OpenAI embeddings API specification. LangChain's OpenAIEmbeddings wrapper abstracts this HTTP communication, providing methods like embed_query() (for single-query embedding) and embed_documents() (for batch document embedding). [40,53] The llama.cpp server handles request batching internally, optimizing GPU utilization by processing multiple embeddings concurrently when batch sizes exceed single-request thresholds.

4.2.2.3 Transcript Processing

Figure 4.5 shows the document processing pipeline for a single transcript. The pipeline consists of three stages: (1) parsing the SRT file into structured chunks using the SRT parser (described in Section 4.2.1), (2) creating LangChain Document objects with meta-

data via the document processor, and (3) adding a transcript_id field to each document's metadata for efficient filtering during retrieval. For simplicity, the transcript_id is derived from the filename by removing the .srt extension and replacing periods with underscores, producing clean identifiers like "example_transcript_1" that can be used in Chroma metadata filters.

```
def process_transcript(self, transcript_path: str) -> List[Document]:

    filename = os.path.basename(transcript_path)
    print(f"Processing {filename}...")

    # Parse SRT file into chunks
    chunks = parse_srt_chunks(transcript_path)
    print(f"  Parsed {len(chunks)} SRT entries")

    # Create documents (with merging if enabled)
    documents = create_documents_from_srt_chunks(
        chunks,
        file_path=filename
    )

    # Add transcript_id metadata (cleaned filename without extension)
    transcript_id = filename.replace('.srt', '').replace('.', '_')
    for doc in documents:
        doc.metadata['transcript_id'] = transcript_id

    print(f"  Created {len(documents)} document chunks")
    return documents
```

Figure 4.5: Document processing pipeline

This processing stage integrates with the chunking logic described in Section 4.2.1, where SRT entries are merged based on token counts to produce chunks of appropriate size for embedding. The resulting Document objects contain both page_content (the merged chunk text) and metadata (chunk_id, source filename, timestamps, speaker information, and transcript_id), enabling both semantic retrieval and metadata-based filtering.

4.2.2.4 Batch Embedding Generation and Vector Store Creation

Embedding generation is the most computationally expensive component of the index building pipeline, requiring forward passes through the transformer-based embedding model for every document chunk. To maximize throughput and GPU utilization, embeddings are generated in batches rather than one-by-one. Figure 4.6 shows the vector store creation logic using Chroma's from_documents() method, which automatically batches embedding requests and stores the resulting vectors in the SQLite-backed database. [41, 42]

```python
# Create vector store
print(f"  Creating Chroma vector store (this will take a while)...")

vector_store = Chroma.from_documents(
    documents=all_documents,
    embedding=self.embeddings,
    collection_name=self.collection_name,
    persist_directory=self.global_store_path
)

print("  ✓ Vector store created and persisted")
```

Figure 4.6: Vector store creation from documents

Figure 4.7 demonstrates incremental document addition to an existing vector store. The add_documents() method extends the existing collection with new embeddings without requiring a full rebuild, preserving the existing embeddings and metadata while appending the new vectors. The optional ids parameter enables upsert behavior (insert if new, update if exists) when document IDs are provided, though the deduplication logic described earlier typically ensures that only new documents reach this stage.

```python
print(f"\n  Adding {len(all_new_documents)} new chunks to vector store...")

# Pass explicit IDs to enable upsert behavior (prevent duplicates)
ids = [doc.id for doc in all_new_documents if hasattr(doc, 'id') and doc.id]
if ids and len(ids) == len(all_new_documents):
    vector_store.add_documents(all_new_documents, ids=ids)
else:
    vector_store.add_documents(all_new_documents)

print("  ✓ New documents added to vector store")
```

Figure 4.7: Incremental document addition

LangChain's automatic batching mechanism abstracts the complexity of batch size tuning, request pipelining, and error handling, enabling the index builder to focus on higher-level logic (deduplication, metadata management) rather than low-level HTTP optimization. However, this abstraction does limit control over batch sizes and parallelism; for production deployments with strict latency or throughput requirements, a custom batching implementation with explicit concurrency control may provide better performance.

4.2.2.5 ChromaDB Storage and Persistence

Chroma stores all embeddings, metadata, and collection configuration in a single SQLite database file (chroma.sqlite3) located in the persist directory. [42] Figure 4.8 illustrates the directory structure, showing how the vector store, BM25 index, and phonetic index are co-located within the same storage hierarchy. This unified structure simplifies backup, replication, and deployment: copying the vector_stores/global_combined/ directory to another machine produces a fully functional index without requiring configuration changes or re-embedding.

```
vector_stores/global_combined/
├── chroma.sqlite3              # Main vector database (embeddings + metadata)
├── phonetic_index.db           # Phonetic search index (SQLite)
└── bm25_index/
    └── bm25_index.pkl          # Serialized BM25 index
```

Figure 4.8: ChromaDB storage directory structure

The embedding generation and vector storage subsystem implements the foundation for the hybrid retrieval pipeline described in Section 4.2.3, providing the semantic signal that complements keyword-based, phonetic, and fuzzy matching. By embedding both queries and documents in the same semantic space, the retriever enables the system to retrieve relevant chunks even when query terms do not appear verbatim in the document text, capturing paraphrasing, synonym variation, and conceptual similarity that lexical methods cannot detect.

4.2.3 Multi-Stage Retrieval Process

The retrieval pipeline implements a four-stage progressive refinement strategy, each stage narrowing the candidate set with increasing computational cost and precision.

4.2.3.1 Stage 1: Hybrid Search

Vector Search (Semantic)

Vector search implements semantic retrieval using dense embedding representations generated by the Jina embedding model. Each document chunk is encoded as a high-dimensional vector (1024 dimensions for jina-embeddings-v3), and retrieval is performed by computing cosine similarity between the query embedding and all document embeddings in the vector store. [35] This approach captures semantic relationships beyond exact keyword matches, enabling the system to retrieve relevant chunks even when queries are paraphrased, use synonyms, or express concepts differently from the source text.

The core vector search function, shown in Figure 4.9, provides a unified interface for semantic retrieval with optional metadata filtering. When a Chroma filter is provided, it restricts the search space to specific transcripts by matching metadata fields, enabling transcript-scoped search. Without filters, the system performs global search across all indexed documents.

```python
def run_semantic_search(
    vector_retriever,
    query: str,
    k: int,
    chroma_filter: Optional[Dict] = None
) -> List[Document]:

    if chroma_filter:
        return vector_retriever.vectorstore.similarity_search(
            query, k=k, filter=chroma_filter
        )
    else:
        return vector_retriever.invoke(query)
```

Figure 4.9: Vector search core function showing semantic retrieval with optional Chroma metadata filtering

Figure 4.10 demonstrates how transcript filters are constructed for the Chroma vector database. The system supports both single-transcript filtering using the $eq operator and multi-transcript filtering using the $in operator. This metadata-based filtering ensures efficient scoping of search results without requiring separate vector indices per transcript.

```python
def _build_chroma_filter(
    transcript_filters_set: Optional[Set[str]]
) -> Optional[Dict]:

    if not transcript_filters_set:
        return None

    if len(transcript_filters_set) == 1:
        # Single transcript filter
        return {"transcript_id": {"$eq": next(iter(transcript_filters_set))}}
    else:
        # Multiple transcripts filter
        return {"transcript_id": {"$in": list(transcript_filters_set)}}
```

Figure 4.10: Chroma metadata filter construction for transcript scoping

Key advantages of vector search include robust handling of paraphrasing, synonym variation, and semantic similarity beyond surface-level keyword matching. It effectively captures conceptual relationships and topical relevance, making it particularly strong for queries that express intent in different words than the source transcript. However, vector search can struggle with proper nouns, rare technical terms, and exact keyword requirements, which motivates the complementary use of BM25 keyword search.

BM25 Search (Keyword)

BM25 implements probabilistic keyword-based retrieval using the BM25Okapi ranking function, a well-established algorithm in information retrieval that scores documents based on term frequency (TF) and inverse document frequency (IDF). [16, 43] Unlike

semantic search, which relies on dense vector representations, BM25 performs exact lexical matching, making it particularly effective for queries containing proper nouns, technical terminology, product names, and specific keywords that must appear verbatim in retrieved chunks.

The BM25 retrieval pipeline consists of five main stages: tokenization, index initialization, query processing, scoring and ranking, and persistence. Each stage is implemented as a modular component within the BM25Retriever class.

Tokenization Strategy

Figure 4.11 shows the default tokenizer implementation, which applies a simple lowercase split on whitespace. This lightweight approach avoids dependencies on NLP libraries while providing adequate performance for most queries.

```
@staticmethod
def _default_tokenizer(text: str) -> List[str]:

    return text.lower().split()
```

Figure 4.11: BM25 tokenization function using lowercase split

Index Initialization

The BM25 index is constructed once and persisted alongside the vector store. It is constructed by tokenizing all document chunks in the corpus and building an inverted index structure, as shown in Figure 4.12. The initialization process iterates over all LangChain Document objects, applies the tokenizer to each document's page content, and passes the tokenized corpus to the BM25Okapi constructor. The BM25Okapi class internally builds an inverted index mapping terms to document IDs and computes IDF weights for each unique term in the corpus. [43] This preprocessing step is performed once during index building and enables fast scoring at query time.

```python
def __init__(
    self,
    documents: List[Document],
    tokenizer: Optional[Callable[[str], List[str]]] = None
):

    if not documents:
        raise ValueError("Cannot create BM25 retriever with empty document list")

    self.documents = documents
    self.tokenizer = tokenizer or self._default_tokenizer

    # Tokenize all documents for BM25 indexing
    print(f"Tokenizing {len(documents)} documents for BM25 indexing...")
    tokenized_corpus = [
        self.tokenizer(doc.page_content)
        for doc in documents
    ]

    # Build BM25 index
    print("Building BM25 index...")
    self.bm25 = BM25Okapi(tokenized_corpus)
    print(f"✓ BM25 index created with {len(documents)} documents")
```

Figure 4.12: BM25 index initialization with corpus tokenization

Query Processing and Metadata Filtering

At query time, the BM25 retriever tokenizes the input query using the same tokenization logic applied during indexing to ensure consistency. The retrieval function supports optional metadata-based filtering to restrict results to specific transcripts, as demonstrated in Figure 4.13. When a filter is provided, the system first identifies document indices matching the specified metadata field and value (e.g., transcript_id), then invokes the optimized get_batch_scores method to compute BM25 scores only for the filtered subset of documents. This selective scoring strategy significantly reduces computational cost when searching within a single transcript or small transcript subset, avoiding unnecessary score computation for out-of-scope documents.

```python
# Tokenize query
tokenized_query = self.tokenizer(query)

if filter_value:
    # Normalize filter_value to list for consistent handling
    if isinstance(filter_value, str):
        filter_values = [filter_value]
    else:
        filter_values = filter_value

    # Filter documents by specified field before ranking
    filtered_indices = [
        i for i, doc in enumerate(self.documents)
        if doc.metadata.get(filter_field) in filter_values
    ]

    if not filtered_indices:
        # No documents match the filter
        return []

    # Get BM25 scores efficiently for only the subset of documents
    # get_batch_scores returns a list of scores corresponding to doc_ids
    scores = self.bm25.get_batch_scores(tokenized_query, filtered_indices)

    # Zip indices and scores together
    filtered_scores = list(zip(filtered_indices, scores))
```

Figure 4.13: BM25 query tokenization and metadata filtering

Scoring and Ranking

After computing BM25 scores for candidate documents, the retriever ranks results by score in descending order and selects the top-k highest-scoring chunks, as shown in Figure 4.14. When metadata filtering is applied, the system sorts the filtered scores and returns the top-k results. Without filtering, it uses NumPy's argsort operation to efficiently identify the top-k document indices from the full score array. The BM25Okapi scoring function applies the standard BM25 formula with parameters k1=1.5 (controlling term frequency saturation) and b=0.75 (controlling document length normalization). [16, 43] These parameter values are widely used defaults in the information retrieval literature and provide a reasonable balance between precision and recall across diverse query types.

```python
# Sort by score (descending) and take top-k
filtered_scores.sort(key=lambda x: x[1], reverse=True)
top_k_results = filtered_scores[:k]
```

Figure 4.14: BM25 scoring and top-k ranking

Once the top-k document indices and scores are determined, the retriever constructs LangChain Document objects by copying the original document content and metadata, then appending the BM25 score as an additional metadata field, as illustrated in Figure 4.15. This metadata preservation ensures that downstream components (e.g., RRF fusion,

rerankers) can access both the original chunk metadata (timestamps, speakers, transcript IDs) and the BM25 relevance score.

```
# Create result documents
results = []
for idx, score in top_k_results:
    doc = self.documents[idx]
    result_doc = Document(
        page_content=doc.page_content,
        metadata={
            **doc.metadata,
            'bm25_score': float(score)
        }
    )
    results.append(result_doc)

return results
```

Figure 4.15: BM25 result document construction

Index Persistence

To avoid rebuilding the BM25 index on every system restart, the retriever implements pickle-based serialization for persistent storage, as shown in Figure 4.16. The save_index method serializes the entire retriever state, including the document list, BM25 index structure, and tokenizer function, to disk. On subsequent system startups, the load_index classmethod deserializes the saved state, enabling fast index loading without re-tokenizing the corpus or recomputing IDF weights. This persistence mechanism significantly reduces startup time for large document collections.

```
def save_index(self, filepath: str):

    # Create parent directory if it doesn't exist
    Path(filepath).parent.mkdir(parents=True, exist_ok=True)

    # Serialize retriever state
    with open(filepath, 'wb') as f:
        pickle.dump({
            'documents': self.documents,
            'bm25': self.bm25,
            'tokenizer': self.tokenizer
        }, f)

    print(f"✓ BM25 index saved to {filepath}")
```

Figure 4.16: BM25 index persistence via pickle serialization

Key advantages of BM25 search include strong performance on exact keyword queries, robust handling of proper nouns and technical terminology, and computational efficiency relative to neural retrieval models. BM25 excels when query terms must appear verbatim

in the retrieved context, making it complementary to vector search, which prioritizes semantic similarity over exact lexical matches.

Phonetic Search (ASR Error Handling)

Phonetic search addresses a critical limitation of exact lexical matching in ASR-generated transcripts: proper nouns, technical terms, and rare vocabulary are frequently misrecognized by speech-to-text models, resulting in transcription errors that preserve phonetic similarity but differ in spelling. For example, a query for "Samsung" may fail to retrieve chunks where the ASR system transcribed the same spoken word as "Samson" or "Samsang." Since these variants share similar pronunciation, they should ideally be retrieved despite lexical differences. Phonetic search solves this problem by encoding words into phonetic representations using the Double Metaphone algorithm, enabling sound-alike matching that is robust to ASR transcription errors. [27, 44]

Database Schema and Index Structure

The phonetic indexer is implemented using a SQLite-backed database with a single primary table and three supporting indices, as shown in Figure 4.17. The phonetic_codes table stores mappings from phonetic codes (e.g., "SMSNK" for "Samsung") to chunk IDs, transcript IDs, and original words. Each row represents one word in one chunk, encoded by its phonetic representation. The composite primary key on (phonetic_code, chunk_id, word) ensures that duplicate entries are automatically deduplicated during batch insertion. Three indices—idx_phonetic_code, idx_transcript, and idx_chunk_id—enable fast lookup by phonetic code (the primary access pattern), filtering by transcript ID, and reverse lookup by chunk ID, respectively.

```sql
-- Main table: stores phonetic code mappings
CREATE TABLE IF NOT EXISTS phonetic_codes (
    phonetic_code TEXT NOT NULL,
    chunk_id INTEGER NOT NULL,
    transcript_id TEXT NOT NULL,
    word TEXT NOT NULL,
    PRIMARY KEY (phonetic_code, chunk_id, word)
);

-- Index for fast phonetic code lookup
CREATE INDEX IF NOT EXISTS idx_phonetic_code
ON phonetic_codes(phonetic_code);

-- Index for transcript filtering
CREATE INDEX IF NOT EXISTS idx_transcript
ON phonetic_codes(transcript_id);

-- Index for chunk ID lookup
CREATE INDEX IF NOT EXISTS idx_chunk_id
ON phonetic_codes(chunk_id);
```

Figure 4.17: Phonetic index database schema with SQLite table and indices

This schema design prioritizes retrieval efficiency: queries typically access the index by

phonetic code (e.g., "find all chunks containing code 'SMSNK'"), which is accelerated by the idx_phonetic_code index. The transcript_id field supports metadata filtering, allowing the system to restrict phonetic search to a specific transcript or transcript set without scanning unrelated chunks. The word field preserves the original token for debugging and analysis, though it is not used in the core retrieval logic.

Tokenization and Word Filtering

Before phonetic encoding, the indexer tokenizes input text by extracting alphabetic words and filtering short tokens, as illustrated in Figure 4.18. The tokenize_and_encode method applies a regular expression ([a-zA-Z]+) to extract contiguous alphabetic sequences, discarding punctuation, numbers, and whitespace. Words shorter than 3 characters are filtered out because short words (e.g., "is", "at", "on") are too common to provide discriminative phonetic matches and would introduce noise into the index. This length threshold balances index size (excluding uninformative short words) against recall (retaining meaningful terms).

```
def tokenize_and_encode(self, text: str) -> List[Tuple[str, str]]:

    # Lowercase and extract words (alphabetic only)
    words = re.findall(r'[a-zA-Z]+', text.lower())

    # Filter short words (< 3 chars) - they're too common for phonetic matching
    words = [w for w in words if len(w) >= 3]
```

Figure 4.18: Phonetic word tokenization and filtering

Double Metaphone Encoding

Figure 4.19 shows the core phonetic encoding logic, which applies the Double Metaphone algorithm to each filtered word. The doublemetaphone function (from the metaphone library) returns a tuple of two phonetic codes: a primary code and an optional alternate code. [27,44] The implementation extracts only the primary code, as testing revealed that the alternate code provided minimal additional recall while significantly increasing index size. The phonetic pairs are deduplicated using a seen set to prevent redundant entries when the same word appears multiple times in a document.

```python
    # Compute phonetic codes
    phonetic_pairs = []
    seen = set()

    for word in words:
        # Get primary phonetic code (ignore alternate)
        code, _ = doublemetaphone(word)

        if code and (code, word) not in seen:
            phonetic_pairs.append((code, word))
            seen.add((code, word))

    return phonetic_pairs
```

Figure 4.19: Double Metaphone phonetic encoding

The Double Metaphone algorithm is particularly well-suited for ASR error handling because it produces consistent codes for homophones and phonetically similar words. [27] For example, "Samsung", "Samson", and "Samsang" produce phonetically similar codes, enabling retrieval across spelling variations. Similarly, other brand names like "Apple" and "Aple" would also produce matching phonetic codes. Figure 4.20 provides additional examples of phonetic encodings.

```
"Samsung" → "SMSNK"
"Samson" → "SMSN"
```

Figure 4.20: Phonetic encoding examples for homophone matching

Index Building and Batch Insertion

The index_documents method, shown in Figure 4.21, constructs the phonetic index by iterating over all document chunks, encoding their content, and batch-inserting the resulting phonetic codes into the SQLite database. The force_rebuild parameter allows the index to be cleared and rebuilt from scratch, useful when the tokenization logic or encoding algorithm changes. In normal operation, the indexer performs incremental updates by inserting only new documents, relying on the "INSERT OR IGNORE" SQL command to skip duplicate entries that match the primary key.

```python
def index_documents(
    self,
    documents: List[Document],
    force_rebuild: bool = False
):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    try:
        if force_rebuild:
            print("  Clearing existing phonetic index...")
            cursor.execute("DELETE FROM phonetic_codes")

        # Prepare batch insert
        records = []

        for doc in documents:
            chunk_id = doc.metadata.get('chunk_id')
            transcript_id = doc.metadata.get('transcript_id', 'unknown')

            if chunk_id is None:
                continue

            # Get phonetic codes for this document
            phonetic_pairs = self.tokenize_and_encode(doc.page_content)

            # Add to batch
            for code, word in phonetic_pairs:
                records.append((code, chunk_id, transcript_id, word))

        # Batch insert all records
        cursor.executemany(
            "INSERT OR IGNORE INTO phonetic_codes VALUES (?, ?, ?, ?)",
            records
        )

        conn.commit()
        print(f"  Added {len(records)} new phonetic entries")

    finally:
        conn.close()
```

Figure 4.21: Phonetic index building with batch insertion

Batch insertion significantly improves indexing performance compared to row-by-row insertion, as it reduces the number of SQL transactions and database commits.

Query Processing and Metadata Filtering

At query time, the phonetic indexer encodes the query into phonetic codes using the same tokenize_and_encode method applied during indexing, then executes a SQL query to find all chunks containing any of the query codes. Figure 4.22 shows the SQL query construction logic for the filtered case, where transcript IDs are provided. The query uses the IN operator to match any query code and any transcript ID, then groups results by chunk_id to count the number of matching codes per chunk. Chunks with higher overlap

counts are ranked higher, as they contain more phonetic matches with the query. The ORDER BY overlap_count DESC clause ensures that the most relevant chunks appear first in the result list.

```
# Build query with optional transcript filter
if transcript_filters:
    # With filter (single or multiple transcripts)
    code_placeholders = ','.join('?' * len(query_codes))
    transcript_placeholders = ','.join('?' * len(transcript_filters))
    sql = f"""
        SELECT chunk_id, COUNT(*) as overlap_count
        FROM phonetic_codes
        WHERE phonetic_code IN ({code_placeholders})
          AND transcript_id IN ({transcript_placeholders})
        GROUP BY chunk_id
        ORDER BY overlap_count DESC
    """
    params = query_codes + transcript_filters
```

Figure 4.22: Phonetic retrieval SQL query with transcript filtering

Figure 4.23 shows the unfiltered case, where no transcript restriction is applied. The query structure is similar but omits the transcript_id filter clause. Both queries support an optional LIMIT clause to restrict results to the top-k chunks, reducing result set size and improving response latency.

```
else:
    # No filter
    placeholders = ','.join('?' * len(query_codes))
    sql = f"""
        SELECT chunk_id, COUNT(*) as overlap_count
        FROM phonetic_codes
        WHERE phonetic_code IN ({placeholders})
        GROUP BY chunk_id
        ORDER BY overlap_count DESC
    """
    params = query_codes

# Add limit if specified
if top_k is not None:
    sql += f" LIMIT {top_k}"
```

Figure 4.23: Phonetic retrieval SQL query without filtering

Score Conversion and Document Retrieval

The phonetic_indexer.retrieve method returns a list of (chunk_id, overlap_count) tuples. To integrate with the main retrieval pipeline, these scores must be converted into LangChain Document objects. Figure 4.24 shows the run_phonetic_search wrapper

function, which calls the phonetic indexer, converts the (chunk_id, count) tuples into a scores dictionary, builds a Chroma metadata filter if transcript filtering is required, and delegates to the _convert_scores_to_ranked_list utility to fetch the actual Document objects from the vector store. This utility function (not shown in detail) queries Chroma by chunk_id to retrieve the full document content and metadata, then sorts the results by phonetic score.

```
def run_phonetic_search(
    phonetic_indexer,
    query: str,
    vector_store,
    k: int,
    transcript_filters: Optional[List[str]] = None
) -> List[Document]:

    # Get phonetic matches from indexer
    phonetic_results = phonetic_indexer.retrieve(
        query,
        transcript_filters=transcript_filters,
        top_k=k
    )

    # Convert to scores dict
    phonetic_scores = {chunk_id: count for chunk_id, count in phonetic_results}

    # Build chroma filter for document fetching
    chroma_filter = None
    if transcript_filters:
        if len(transcript_filters) == 1:
            chroma_filter = {"transcript_id": {"$eq": transcript_filters[0]}}
        else:
            chroma_filter = {"transcript_id": {"$in": transcript_filters}}

    # Convert scores to ranked document list
    return _convert_scores_to_ranked_list(
        phonetic_scores,
        vector_store,
        chroma_filter
    )
```

Figure 4.24: Phonetic score conversion and document retrieval

Key advantages of phonetic search include robust handling of ASR transcription errors, particularly for proper nouns, product names, and technical terms that are frequently misrecognized. The SQLite-backed index provides efficient lookup with logarithmic complexity for code-based queries, and the overlap scoring mechanism naturally ranks chunks with higher phonetic similarity. This approach is complementary to BM25 (which requires exact lexical matches) and vector search (which may not distinguish between phonetically similar terms). Phonetic search is particularly effective when users know the approximate pronunciation of a term but are uncertain of the exact spelling used in the transcript.

Fuzzy Matching (Rare Token Typos)

Fuzzy matching provides a complementary approach to phonetic search by using edit-

distance similarity to detect and correct typographical errors and spelling variations in rare tokens. While phonetic search maps phonetically similar words to the same encoded representation, fuzzy matching directly computes string similarity using the Levenshtein edit distance, enabling it to handle character-level perturbations such as insertions, deletions, substitutions, and transpositions. [28] This makes fuzzy matching particularly effective for cases where ASR errors result in lexically close but phonetically distinct transcriptions (e.g., "Samsung" vs "Samsang", "GPT4" vs "GPT-4").

However, applying fuzzy matching indiscriminately across all query tokens is computationally prohibitive, as it requires pairwise edit-distance computation between query tokens and all tokens in all candidate documents. Furthermore, most common words (e.g., "the", "said", "meeting") are unlikely to contain typos and are already well-covered by semantic and keyword search. To balance recall and computational cost, the implementation adopts a selective fuzzy matching strategy: only rare tokens are subjected to fuzzy matching, while common tokens are excluded from this expensive matching process.

Rare Token Detection Using Zipf Frequency

Figure 4.25 shows the rare token detection logic, which uses the Zipf frequency metric from the wordfreq library to classify tokens as common or rare. [46] The Zipf scale represents word frequency as the base-10 logarithm of occurrences per billion words in a large corpus, providing an intuitive scale where high values indicate common words and low values indicate rare words. [46] The threshold is set at 4.0, corresponding to approximately 1 occurrence per 100,000 words. Words with Zipf frequency below 4.0 are classified as rare and subjected to fuzzy matching, while words above the threshold are considered too common to benefit from this expensive operation. Words not found in the wordfreq dictionary (e.g., proper nouns, technical jargon) return a frequency of 0.0 and are automatically classified as rare, which is the desired behavior for ASR error handling.

```python
def is_rare_token(self, token: str) -> bool:

    # Get Zipf frequency (log10 of occurrences per billion words)
    # 0.0 means not in dictionary (infinite rarity)
    freq = zipf_frequency(token, 'en')

    # Threshold 4.0 corresponds to 10^4 per billion (1 in 100,000)
    # Common words > 4.0, Rare words < 4.0
    return freq < 4.0
```

Figure 4.25: Rare token detection using Zipf frequency

The Zipf threshold of 4.0 was selected based on empirical testing and represents a reasonable trade-off between recall and computational cost. Common words like "the" (Zipf $\approx$ 7.8), "meeting" (Zipf $\approx$ 5.2), and "cat" (Zipf $\approx$ 4.6) are excluded from fuzzy matching, as they rarely contain errors and do not benefit from edit-distance correction. In contrast, rare terms like "Samsung" (Zipf $\approx$ 3.8), "iPhone" (Zipf $\approx$ 3.5), and "GPT4" (Zipf $=$ 0.0) are flagged for fuzzy matching, as these are prone to ASR misrecognition and typos.

Extracting Rare Tokens from Query

Figure 4.26 demonstrates the extraction of rare tokens from the input query. The extract_rare_tokens method first tokenizes the query using a simple regular expression to extract alphanumeric word sequences, then filters the token list by applying the is_rare_token predicate to each token. Only tokens classified as rare are returned, effectively reducing the fuzzy matching workload by excluding common words. If no rare tokens are detected in the query, the fuzzy matching pipeline short-circuits and returns an empty result list, avoiding unnecessary computation.

```
def extract_rare_tokens(self, query: str) -> List[str]:

    # Tokenize (extract alphanumeric words)
    tokens = re.findall(r'\w+', query)

    # Filter to rare tokens
    rare_tokens = [t for t in tokens if self.is_rare_token(t)]

    return rare_tokens
```

Figure 4.26: Extracting rare tokens from query

For example, the query "Samsung phone meeting" would extract only "Samsung" as a rare token, while "phone" and "meeting" would be filtered out. This selective approach ensures that fuzzy matching focuses computational resources on the terms most likely to benefit from error correction.

Fetching the Candidate Document Pool

Unlike vector search and BM25, which return ranked candidate lists, fuzzy matching operates over a full document pool within the specified scope (e.g., filtered by transcript ID). Figure 4.27 shows how the candidate pool is fetched using the _fetch_full_document_pool utility, which retrieves all documents matching the optional Chroma metadata filter. This broad candidate set is necessary because fuzzy matching does not rely on pre-ranked results from other retrievers; instead, it independently computes edit-distance scores for all documents in scope.

```
# Extract rare tokens from query
rare_tokens = fuzzy_matcher.extract_rare_tokens(query)

# Early exit if no rare tokens
if not rare_tokens:
    return []

# Fetch full document pool in scope
all_candidate_docs = _fetch_full_document_pool(vector_store, chroma_filter)

if not all_candidate_docs:
    return []
```

Figure 4.27: Fetching candidate document pool for fuzzy matching

The _fetch_full_document_pool helper function uses a large top-k value (e.g., 10,000) to retrieve all documents within the filter scope, effectively treating the similarity_search call as a "fetch all" operation. While this approach may seem inefficient, it is mitigated by the fact that fuzzy matching only activates when rare tokens are present, which occurs in a minority of queries.

Fuzzy Scoring Algorithm: Pairwise Edit-Distance Computation

The core fuzzy scoring algorithm, shown in Figures 4.28 and 4.29, iterates over all candidate documents and computes the maximum edit-distance similarity between query rare tokens and document tokens. For each document, the algorithm tokenizes the document content, then performs a nested loop over query rare tokens and document tokens, computing the fuzzy ratio using the RapidFuzz library. The fuzz.ratio function returns an integer score in the range [0, 100] representing percentage similarity, which is normalized to a [0, 1] scale by dividing by 100. The maximum score across all query-document token pairs is retained as the document's fuzzy score.

```python
def compute_fuzzy_scores(
    self,
    query: str,
    candidate_pool: List[Document],
    top_k: Optional[int] = None
) -> Dict[int, float]:

    # Extract rare tokens from query
    rare_tokens = self.extract_rare_tokens(query)

    if not rare_tokens:
        return {}

    # Compute scores
    scores = {}
```

Figure 4.28: Fuzzy scoring algorithm - setup and initialization

```python
    for doc in candidate_pool:
        chunk_id = doc.metadata.get('chunk_id')
        if chunk_id is None:
            continue

        doc_tokens = self._tokenize_document(doc)
        if not doc_tokens:
            continue

        # Compute best match score
        max_score = 0.0
        for query_token in rare_tokens:
            for doc_token in doc_tokens:
                # Compute edit-distance similarity using RapidFuzz
                score = fuzz.ratio(
                    query_token.lower(),
                    doc_token.lower()
                ) / 100.0  # Normalize to 0-1 scale

                max_score = max(max_score, score)

                # Early exit if we found a near-perfect match
                if max_score >= 0.95:
                    break

            if max_score >= 0.95:
                break

        # Only include if above threshold
        if max_score >= self.threshold:
            scores[chunk_id] = max_score
```

Figure 4.29: Fuzzy scoring algorithm - nested loop with RapidFuzz ratio

An important optimization is the early exit logic: if the maximum score reaches 0.95 (indicating a near-perfect match), the inner loops terminate immediately, avoiding redundant comparisons. This optimization significantly reduces computation time when high-quality matches are found early in the loop. Only documents with fuzzy scores exceeding the configured threshold (default: 0.8) are included in the result set, filtering out low-quality matches that are unlikely to be relevant.

The RapidFuzz library is used for edit-distance computation because it provides highly optimized implementations of the Levenshtein distance algorithm, significantly outperforming pure Python alternatives. [28, 45] Example fuzzy ratios include: "Samsung" vs "Samsang" (93% similarity), "Samsung" vs "Samson" (86% similarity), and "GPT4" vs "GPT-4" (80% similarity).

Top-K Selection and Ranking

After computing fuzzy scores for all candidate documents, the algorithm sorts the results by score in descending order and selects the top-k matches, as shown in Figure 4.30. This ranking step ensures that only the most relevant fuzzy matches are returned, preventing the retrieval of marginally similar chunks that may introduce noise into the final fused

result set.

```
    # Sort by score and limit to top_k
    if top_k is not None and len(scores) > top_k:
        sorted_items = sorted(
            scores.items(),
            key=lambda x: x[1],
            reverse=True
        )[:top_k]
        scores = dict(sorted_items)

    return scores
```

Figure 4.30: Fuzzy top-k selection and ranking

Phonetic search and fuzzy matching address ASR-induced errors through complementary mechanisms. Phonetic search uses Double Metaphone encoding to efficiently retrieve homophones and sound-alike terms via fast indexed lookup, making it effective for cases where the transcription error preserves pronunciation (e.g., "Samsung" vs "Samson"). In contrast, fuzzy matching relies on edit-distance similarity to correct character-level perturbations, making it effective for typos, hyphenation differences, and cases where the error is lexical rather than phonetic (e.g., "GPT4" vs "GPT-4", "iPhone" vs "i Phone").

Phonetic search is applied broadly to all tokens of length ≥ 3 characters, as the SQLite-backed index enables efficient lookup with logarithmic complexity. Fuzzy matching is restricted to rare tokens (Zipf < 4.0) because pairwise edit-distance computation scales poorly with candidate set size, making it computationally expensive. Together, these signals improve retrieval robustness under noisy transcripts by capturing both phonetic equivalence and character-level perturbations, providing complementary coverage of the ASR error space.

Integration: Parallel Multi-Signal Retrieval

The four retrieval methods described above are executed in parallel during Stage 1 of the retrieval pipeline. Parallel execution is feasible because the four signals operate independently: each method accesses its own index and produces a ranked list of documents without requiring input from other retrievers. This independence enables concurrent execution, reducing overall retrieval latency compared to sequential execution.

In practice, Python's Global Interpreter Lock (GIL) limits the effectiveness of thread-based parallelism for CPU-bound tasks, so the current implementation executes methods sequentially but is structured to support parallel execution via multiprocessing or asynchronous I/O in future optimizations.

Summary: Search Method Characteristics

Table 4.3 summarizes the key characteristics of all four search methods implemented in Stage 1, including their algorithms, index types, scoring metrics, default top-k values, and primary use cases. This comparison highlights the complementary nature of the four signals: vector search excels at semantic similarity, BM25 at exact keyword matching,

phonetic search at handling ASR transcription errors, and fuzzy matching at correcting typos in rare terms. The varying index types reflect different performance trade-offs: vector search and phonetic search use persistent database-backed indices for fast lookup, BM25 uses an inverted index for maximum speed, and fuzzy matching performs full scans over the candidate pool to compute edit-distance similarity.

Search Method	**Algorithm**	**Index Type**	**Scoring Metric**	**Top-K**	**Best For**
Vector Search	Cosine similarity over embeddings	Dense vectors (Chroma)	Embedding distance	10	Semantic similarity, paraphrasing
BM25 Search	Probabilistic TF-IDF ranking	Inverted index	BM25 score	25	Keyword matching, proper nouns
Phonetic Search	Double Metaphone encoding	SQLite phonetic index	Overlap count	10	ASR errors, homophones
Fuzzy Matching	Edit-distance (RapidFuzz)	No index (full scan)	Fuzzy ratio	10	Typos, spelling variations

Table 4.3: Search method characteristics comparing algorithm, indexing strategy, scoring approach, default retrieval limits, and primary use cases

The default top-k values are tuned based on the signal's role in the hybrid pipeline. Vector search and phonetic search retrieve 10 chunks each, providing a focused set of semantically or phonetically relevant results. BM25 retrieves more chunks because keyword-based recall is typically lower than semantic recall, requiring a larger initial candidate set. Fuzzy matching also retrieves 10 chunks but only activates when rare tokens are detected, making its contribution selective rather than universal. These top-k values are configurable and can be adjusted during system tuning to balance recall, precision, and computational cost.

4.2.3.2 Stage 2: Reciprocal Rank Fusion (RRF)

The RRF algorithm combines rankings from all four retrieval signals into a unified ranking. Figure 4.31 shows the RRF weights for hybrid retrieval signals:

```
score(d) = Σ weight_i * (1 / (k + rank_i(d)))

where:
  k = 60 (HYBRID_FUSION_K constant from literature)
  weight_i = search method weight (VECTOR_WEIGHT, BM25_WEIGHT, PHONETIC_WEIGHT, FUZZY_WEIGHT)
  rank_i(d) = rank of document d in search method i (1-indexed)

Configured weights:
  VECTOR_WEIGHT = 1.0    (primary signal)
  BM25_WEIGHT = 1.0      (primary signal)
  PHONETIC_WEIGHT = 0.2  (supplementary signal)
  FUZZY_WEIGHT = 0.2     (supplementary signal)
```

Figure 4.31: Reciprocal Rank Fusion weights for hybrid retrieval signals

The RRF approach provides score normalization across heterogeneous retrieval methods, including semantic, keyword, phonetic, and fuzzy signals, and remains robust to inherent score scale differences between these components. Its weighted formulation further enables explicit prioritization of primary retrieval signals (e.g., semantic and BM25) while still incorporating supplementary signals (e.g., phonetic and fuzzy) to improve recall under ASR noise. [10] In addition, RRF is well-established in information retrieval literature, straightforward to implement, and computationally efficient, making it suitable for low-latency retrieval pipelines. Overall, it effectively combines complementary retrieval strengths to produce a more accurate and ASR-robust candidate set for downstream reranking and generation.

4.2.3.3 Stage 3: Cross-Encoder Reranking

Stage 3 applies a cross-encoder neural reranking model to re-score the candidates retrieved from RRF fusion, providing a final precision-focused refinement step. While the retrieval methods in Stage 1 (vector search, BM25, phonetic, fuzzy) and the fusion algorithm in Stage 2 (RRF) cast a wide net to maximize recall, cross-encoder reranking applies a more computationally expensive but significantly more accurate relevance model to select the final top-k chunks for context assembly.

Cross-Encoder vs Bi-Encoder Architecture

The distinction between bi-encoder and cross-encoder architectures is fundamental to understanding why reranking improves retrieval quality. Bi-encoder models (such as the Jina embedding model used for vector search) independently encode the query and documents into fixed-length vectors, then compute similarity via cosine distance. [35] This approach enables fast retrieval through pre-computed document embeddings and efficient vector search, but it lacks the capacity to model fine-grained query-document interactions because the query and document are encoded separately without mutual attention.

Cross-encoder models, in contrast, jointly encode the query and document as a single concatenated input, allowing the transformer layers to model token-level interactions between query terms and document terms. [11] This bidirectional attention mechanism enables the model to capture nuanced relevance signals such as term proximity, phrase matching, and semantic entailment that bi-encoders cannot represent. However, cross-encoders require inference for every query-document pair, making them prohibitively

expensive for initial retrieval over large corpora. The two-stage retrieval architecture (bi-encoder for fast retrieval → cross-encoder for precise reranking) balances efficiency and precision by applying the slow but accurate cross-encoder only to a small candidate set (30-40 chunks from RRF fusion).

Reranker Service Configuration and Initialization

The reranking functionality is implemented as a separate microservice, hosted via llama.cpp server. This separation from the main LLM service and embedding service enables independent scaling and resource allocation for each component.

HTTP Request Construction and Payload Preparation

At reranking time, the system constructs an HTTP POST request containing the query string and the list of candidate document texts. Figure 4.32 shows the payload preparation logic: the page_content field is extracted from each LangChain Document object to create a list of plain text strings, which are then packaged into a JSON payload along with the model name and query. The optional top_n parameter, if provided, is included in the payload to instruct the server to return only the top-n results (though the client enforces this limit locally as well to ensure consistency).

```
# Extract text content from documents
doc_texts = [doc.page_content for doc in documents]

# Prepare request payload
payload = {
    "model": self.model_name,
    "query": query,
    "documents": doc_texts
}

if top_n is not None:
    payload["top_n"] = top_n
```

Figure 4.32: Request payload preparation

Response Parsing and Score-Based Sorting

The reranker service returns a JSON response containing a list of results, where each result includes an index (referring to the position in the input documents array) and a relevance_score (typically in the range [0, 1], with higher scores indicating greater relevance). Figure 4.33 shows the expected response format.

```
{
  "results": [
    {"index": 2, "relevance_score": 0.9523},
    {"index": 0, "relevance_score": 0.8876},
    {"index": 5, "relevance_score": 0.7234},
    {"index": 1, "relevance_score": 0.6891}
  ]
}
```

Figure 4.33: Expected response format

Although the reranker service typically returns results pre-sorted by relevance score, the client explicitly sorts the results to ensure consistency regardless of server implementation, as shown in Figure 4.34. The sorted function orders results by the relevance_score field in descending order (highest scores first), with a fallback to 0 if the score is missing.

```
# Sort results by relevance score (should already be sorted, but ensure it)
ranked_results = sorted(
    result["results"],
    key=lambda x: x.get("relevance_score", 0),
    reverse=True
)
```

Figure 4.34: Sorting by relevance score

Metadata Preservation and Document Reconstruction

After sorting, the system maps the reranked indices back to the original Document objects, preserving all metadata while adding the rerank_score field, as demonstrated in Figure 4.35. For each result, the index is validated to ensure it falls within the bounds of the input documents array, then the corresponding document is retrieved. A new Document object is constructed with the same page_content and a merged metadata dictionary that includes all original fields (chunk_id, transcript_id, timestamps, etc.) plus the new rerank_score. This metadata preservation ensures that downstream components (context formatting, LLM prompts) can access both the original chunk information and the reranking score.

```python
# Map back to original documents
reranked_docs = []
for item in ranked_results:
    idx = item["index"]
    if 0 <= idx < len(documents):
        # Optionally add relevance score to metadata
        doc = documents[idx]
        # Create a copy with updated metadata
        reranked_doc = Document(
            page_content=doc.page_content,
            metadata={
                **doc.metadata,
                "rerank_score": item.get("relevance_score", 0)
            }
        )
        reranked_docs.append(reranked_doc)
```

Figure 4.35: Mapping results to original documents

Finally, the top_n limit is enforced locally by slicing the reranked_docs list, as shown in Figure 4.36. This client-side enforcement is necessary because not all reranking servers respect the top_n parameter in the request payload, and it provides a consistent guarantee that exactly top_n (or fewer, if the input was smaller) documents are returned.

```python
# Enforce top_n slicing (in case server ignores the parameter)
if top_n is not None:
    reranked_docs = reranked_docs[:top_n]

return reranked_docs
```

Figure 4.36: Top-n selection

Key advantages of cross-encoder reranking include significantly improved relevance precision compared to bi-encoder similarity or keyword-based scoring, the ability to capture fine-grained semantic entailment and term proximity signals, and robustness to lexical variation (paraphrasing, synonyms) that BM25 cannot handle. The main trade-off is computational cost: cross-encoder inference requires $O(n)$ forward passes for n candidate documents, whereas bi-encoder similarity requires only a single query encoding plus $O(1)$ vector lookups. The two-stage architecture mitigates this cost by limiting cross-encoder reranking to a small candidate set (30-40 chunks from RRF fusion), keeping reranking latency manageable.

4.2.4 Context Assembly and Prompt Construction

After the retrieval and reranking stages produce a ranked list of relevant document chunks, these chunks must be transformed into a structured context string suitable for LLM consumption. Context assembly involves formatting the retrieved chunks with appropriate

metadata headers, concatenating them in a readable format, and injecting the result into a prompt template that defines the assistant's behavior and constraints. This section describes the implementation of context formatting, prompt template design, and LLM invocation for answer generation.

4.2.4.1 Context Formatting

The context formatting layer transforms raw Document objects into a structured text representation that the LLM can interpret effectively. Figure 4.37 demonstrates the formatting implementation, which processes each document individually and constructs a header containing the chunk ID, timestamp range, and sequential position within the result set. Each chunk is formatted as a header line followed by its content, with chunks separated by double newlines for clear visual separation. The header format provides the LLM with temporal context (start and end times) and positional information (chunk ID and position in ranked list), enabling it to reference specific chunks in its responses if needed.

```python
def format_flat_chunks(documents: List[Document]) -> str:

    context_parts = []
    for i, doc in enumerate(documents, 1):
        chunk_id = doc.metadata.get('chunk_id', 'unknown')
        start_time = doc.metadata.get('start_time', '')
        end_time = doc.metadata.get('end_time', '')

        header = f"[Chunk {i} - ID: {chunk_id}"
        if start_time and end_time:
            header += f", Time: {start_time} -> {end_time}"
        header += "]"

        context_parts.append(f"{header}\n{doc.page_content}")

    return "\n\n".join(context_parts)
```

Figure 4.37: Chunk formatting with individual metadata headers

Figure 4.38 shows an example of the output format for a single document.

```
[Chunk 1 - ID: 42, Time: 00:05:23 -> 00:05:45]
Speaker A discusses the project timeline and key milestones...

[Chunk 2 - ID: 43, Time: 00:05:45 -> 00:06:08]
Speaker B responds with concerns about resource allocation...
```

Figure 4.38: Output Format Example

The timestamp format follows the SRT standard, preserving millisecond precision from the original transcript. This enables users to locate specific segments in the source audio or video if they wish to verify the LLM's answers. The chunk ID field corresponds to

the sequential position of the chunk within its source transcript, allowing for precise cross-referencing with the original document.

Citation Formatting for User Transparency

In addition to formatting context for the LLM, the system provides a citation formatting function that generates structured metadata for user-facing displays, as shown in Figure 4.39. This function returns both the formatted context string and a list of source citations, each containing the chunk ID, timestamp, source filename, transcript ID, and a preview of the chunk content. The citation information enables web interfaces to display retrieval provenance alongside generated answers, allowing users to verify the LLM's responses against the source material and explore the full context if needed.

```python
def format_chunks_with_citations(documents) -> Dict[str, any]:

    context = format_retrieved_chunks(documents)

    # Flatten groups to individual chunks for source citations
    if documents and isinstance(documents[0], dict):
        # Grouped format - flatten for citations
        flat_docs = []
        for group in documents:
            flat_docs.extend(group['chunks'])
        documents = flat_docs

    # Generate citations
    sources = []
    for i, doc in enumerate(documents, 1):
        # Create a preview (first 100 characters)
        preview = doc.page_content[:100]
        if len(doc.page_content) > 100:
            preview += "..."

        citation = {
            'chunk_number': i,
            'chunk_id': doc.metadata.get('chunk_id', 'unknown'),
            'preview': preview,
            'full_text': doc.page_content,
            'source': doc.metadata.get('source', 'unknown'),
            'transcript_id': doc.metadata.get('transcript_id', 'unknown'),
            'start_time': doc.metadata.get('start_time', ''),
            'end_time': doc.metadata.get('end_time', '')
        }
        sources.append(citation)

    return {
        'context': context,
        'sources': sources
    }
```

Figure 4.39: Citation formatting for user transparency and source attribution

The preview field contains the first 100 characters of each chunk, providing users with a quick glimpse of the content without requiring them to expand the full text. The full_text

field preserves the complete chunk content for detailed inspection. This dual-level presentation balances information density with accessibility, enabling users to quickly scan sources while retaining the ability to dive deeper when necessary.

4.2.4.2 Prompt Templates

The prompt template layer defines the structure and content of messages sent to the LLM, including the system message that sets the assistant's role and behavior, the human message that contains the user's question, and placeholders for dynamic content such as retrieved context and conversation history. The system uses LangChain's ChatPromptTemplate abstraction to construct multi-turn conversation prompts with type-safe variable substitution. [31, 54]

System Message Design

Figure 4.40 shows the RAG system message template, which establishes the assistant's role as a helpful agent with access to transcript excerpts and defines explicit constraints on its behavior. The system message includes three key components: (1) a role definition ("You are a helpful assistant"), (2) a context injection placeholder ({retrieved_context}), and (3) behavioral constraints that ground the LLM's responses in the provided context and prohibit hallucination.

```
RAG_SYSTEM_MESSAGE = """
You are a helpful assistant. You have access to relevant
excerpts from a transcript:

{retrieved_context}

Use these excerpts to answer the user's questions. If the
answer cannot be found in the provided excerpts, say so clearly.
Do not make up information not present in the excerpts.
"""
```

Figure 4.40: RAG system message template defining assistant role and constraints

The grounding instruction ("Use these excerpts to answer the user's questions") explicitly directs the LLM to base its responses on the provided context rather than relying solely on parametric knowledge. The transparency instruction ("If the answer cannot be found in the provided excerpts, say so clearly") encourages the LLM to admit when it lacks sufficient information, reducing the risk of plausible but incorrect answers. The hallucination prevention directive ("Do not make up information not present in the excerpts") reinforces these constraints, explicitly forbidding fabrication. This combination of positive instructions (what to do) and negative constraints (what not to do) has proven effective in reducing hallucination rates in empirical testing.

Human Message Template and Reasoning Suppression

Figure 4.41 demonstrates the human message template, which contains the user's question followed by the /no_think directive. This directive is specific to Qwen and similar models that support chain-of-thought reasoning via <think>...</think> tags. [55] When

/no_think is appended to the input, the model suppresses the output of explicit reasoning steps, producing only the final answer. This improves response cleanliness for end users, who typically do not need to see the model's internal reasoning process.

```
RAG_HUMAN_MESSAGE = "{question} /no_think"
```

Figure 4.41: Human message template with reasoning suppression directive

Multi-Turn Chat Prompt Construction

Figure 4.42 shows how the system message and human message templates are combined with a chat history placeholder to create a complete multi-turn conversation prompt. The ChatPromptTemplate.from_messages method constructs a prompt with three components: (1) the system message containing the role definition and context, (2) a placeholder for previous conversation turns, and (3) the current human message with the user's question. [54, 56]

```
from langchain_core.prompts import ChatPromptTemplate

def get_rag_chat_prompt():
    """Create RAG chat prompt template with chat history support."""
    return ChatPromptTemplate.from_messages([
        ("system", RAG_SYSTEM_MESSAGE),
        ("placeholder", "{chat_history}"),
        ("human", RAG_HUMAN_MESSAGE)
    ])
```

Figure 4.42: Complete RAG chat prompt construction with history support

The chat history placeholder enables multi-turn interactions where the LLM can reference previous questions and answers when responding to follow-up queries. For example, if the user asks "What is the project timeline?" followed by "Who is responsible for Phase 2?", the LLM can interpret "Phase 2" in the context of the earlier question about the timeline. This capability is essential for natural conversational interactions, as it eliminates the need for users to repeat context with each query.

Chat History Structure

Figure 4.43 illustrates how chat history is structured using LangChain's message types. Each conversation turn is represented as a HumanMessage (user input) or AIMessage (assistant response), and these messages are accumulated in a list that is injected into the chat_history placeholder. [56] The LLM receives the entire conversation history along with the current question, enabling it to maintain context across multiple turns. This approach supports pronouns, follow-up questions, and clarifications that reference earlier parts of the conversation.

```python
from langchain_core.messages import HumanMessage, AIMessage

# Example chat history structure
chat_history = [
    HumanMessage(content="What is the project timeline?"),
    AIMessage(content="The project timeline spans 6 months..."),
    HumanMessage(content="Who is responsible for Phase 2?"),
    AIMessage(content="Speaker B is responsible for Phase 2...")
]

# Injected into prompt via placeholder
prompt.invoke({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": "When does Phase 2 start?"
})
```

Figure 4.43: Chat history structure for multi-turn conversation support

The chat history is reset when the user starts a new conversation or switches to a different transcript, ensuring that context from unrelated queries does not leak into subsequent interactions. The system limits history length to the most recent N turns (e.g., last 5 exchanges) to prevent the prompt from exceeding the LLM's context window, though this constraint is rarely reached in practice given the short length of typical question-answer pairs.

Streaming Response Handling

Figure 4.44 demonstrates streaming invocation using the chain.stream method. Instead of returning a single string containing the complete response, the stream method returns a generator that yields tokens incrementally as they are produced by the LLM. This enables real-time display in chat interfaces, where each token is printed immediately upon arrival.

```python
# Streaming invocation for real-time display
for chunk in chain.stream({
    "retrieved_context": formatted_context,
    "chat_history": chat_history,
    "question": user_question
}):
    # Print each token as it arrives
    print(chunk, end="", flush=True)

print()  # Newline after complete response
```

Figure 4.44: Streaming response handling for real-time token display

Streaming mode does not reduce total generation time. The LLM still takes the same amount of time to produce the full response, but it significantly improves perceived responsiveness from the user's perspective. Users see the first tokens quickly, providing immediate feedback that the system is working.

4.3 Stretch Goals and Deployment

This section will be completed in the final report, once everything has been finalized.

Chapter 5

Evaluation and Results

This chapter presents the evaluation pipeline implementation, experimental results, and discussion used to assess the retriever system and its end-to-end performance.

5.1 Evaluation Pipeline Implementation

A rigorous evaluation framework was developed to assess both the quality and performance of the RAG system. The evaluation pipeline comprises dataset generation, metric computation, and comparative benchmarking components.

This evaluation framework is designed to operate automatically, eliminating the need for manual human intervention. Traditional evaluation approaches require extensive manual effort, including human annotation of ground truth question-answer pairs, manual inspection of retrieved document relevance, and subjective assessment of generated answer quality [57]. Such manual processes are time-consuming, costly, and difficult to scale across large transcript corpora.

With the advancement of LLM, it is now feasible to design an automated evaluation framework that leverages LLM capabilities for both dataset generation and answer quality assessment [58]. The proposed pipeline automates ground truth generation using an LLM to synthesize contextually relevant question-answer pairs from transcript excerpts (Section 5.1.1), computes retrieval metrics algorithmically through temporal overlap detection (Section 5.1.2), and employs an LLM judge to evaluate semantic correctness of generated answers (Section 5.1.3). This automation enables rapid iteration during system development, facilitates reproducible evaluation across configuration changes, and scales efficiently to accommodate additional transcripts without proportional increases in human labor.

5.1.1 Dataset Generation Methodology

Ground truth evaluation datasets were generated automatically using an LLM-based approach. The generation process follows a structured pipeline that transforms raw SRT transcripts into question-answer pairs with temporal metadata for retrieval evaluation.

5.1.1.1 Context Window Grouping

The grouping algorithm aggregates small SRT chunks into larger context windows suitable for LLM processing. The grouping process employs a greedy first-fit algorithm that iterates through SRT chunks sequentially, accumulating text until a maximum character

threshold is reached.

```python
def group_chunks(chunks: List[Dict], max_chars: int) -> List[Dict]:
    grouped = []
    current_group = {
        "text": "",
        "start_time": chunks[0]["start_time"],
        "end_time": chunks[0]["end_time"],
        "chunk_indices": []
    }
```

Figure 5.1: Context window grouping function signature with metadata tracking

The algorithm maintains temporal boundaries by tracking the start and end timestamps of each grouped window, ensuring that generated questions can be linked back to specific time ranges in the source transcript. This temporal metadata is critical for subsequent retrieval evaluation.

Figure 5.18 illustrates the transformation pipeline from raw SRT transcripts to a grouped window:

```
Chunk 1:
Timestamp: 00:01:30,000 --> 00:01:45,000
Text: "The project timeline spans six months,
       from January to June."

Chunk 2:
Timestamp: 00:01:45,500 --> 00:02:00,200
Text: "We need to complete Phase 1 by the end
       of February."

Chunk 3:
Timestamp: 00:02:00,800 --> 00:02:15,300
Text: "Speaker B is responsible for coordinating
       the deliverables."

           ↓ (Grouping Algorithm)

start_time: 00:01:30,000
end_time: 00:02:15,300
text: "The project timeline spans six months, from
       January to June. We need to complete Phase 1
       by the end of February. Speaker B is
       responsible for coordinating the deliverables."
chunk_indices: [0, 1, 2]

           ↓ (LLM QA Generation)
```

Figure 5.2: Example transformation from raw SRT transcript chunks to a grouped window with temporal metadata

5.1.1.2 LLM API Call for QA Generation

The QA generation leverages DeepSeek's chat API through LangChain's OpenAI-compatible interface. Figure 5.3 shows the chain initialization:

```
def initialize_generator_chain():

    llm = ChatOpenAI(
        model="deepseek-chat",
        openai_api_base=DEEPSEEK_BASE_URL,
        openai_api_key=DEEPSEEK_API_KEY,
        temperature=0.7
    )

    prompt = get_qa_generation_prompt()

    return prompt | llm | StrOutputParser()
```

Figure 5.3: LangChain pipe syntax for QA generation chain

The LangChain pipe operator chains three components: the prompt template, the LLM, and the string output parser. The temperature of 0.7 was selected to balance creativity in question formulation with factual grounding. Lower temperatures (e.g., 0.2) produced repetitive questions, while higher temperatures (e.g., 1.0) occasionally generated questions not directly answerable from the context. Figure 5.4 shows the LLM generation:

```
# Uses DeepSeek API for QA generation
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": prompt}
    ],
    temperature=0.7  # Some creativity for varied questions
)
```

Figure 5.4: LLM API call for QA pair generation with temperature 0.7

5.1.1.3 Generation Prompt Template

The generation prompt template specifies requirements for question-answer pairs. Figure 5.5 shows the prompt template:

```
f"""Based on the following transcript excerpt, generate 2-3 question-answer pairs.

Transcript:
{context}

Requirements:
1. Questions should be specific and answerable from the text
2. Include a mix of detail-oriented and conceptual questions
3. Answers should be concise but complete
4. Questions should be self-contained (no unresolved pronouns)

Output format (JSON without markdown):
[
  {{
    "question": "...",
    "answer": "..."
  }},
  ...
]
"""
```

Figure 5.5: QA generation prompt template with requirements and format

The prompt template enforces three critical requirements. First, questions must be self-contained, avoiding pronouns like "he," "she," or "it" without clear antecedents. This ensures questions can be answered without additional context beyond the retrieved chunks. Second, answers must be directly supported by the provided text, preventing hallucination. Third, the output must be valid JSON, enabling automated parsing and validation.

The template requests 2-3 question-answer pairs per context window, balancing dataset size with question quality. Requesting more pairs per window (e.g., 5-7) resulted in lower-quality questions that repeated similar information.

5.1.1.4 JSON Output Format and Metadata Enrichment

Each generated QA pair is enriched with three metadata fields. The source_start and source_end fields capture the temporal boundaries of the context window from which the question was generated. This temporal metadata enables retrieval relevance checking during evaluation by allowing the system to verify whether retrieved chunks overlap with the ground truth time range. The context field stores the full context window text, enabling manual validation of question quality and answer correctness.

Figure 5.6 shows an example of the ground truth JSON format:

```
[
  {
    "question": "What is the title of the passage?",
    "answer": "The title is 'The North Wind and the Sun'.",
    "source_start": "00:00:00,640",
    "source_end": "00:03:19,180",
    "context": "full text of source context window..."
  }
]
```

Figure 5.6: Ground truth JSON format example with timestamps

Table 5.1 shows the ground truth schema field descriptions:

Field	Type	Description
question	string	Self-contained question derived from context
answer	string	Ground truth answer from source text
source_start	string	Start timestamp in SRT format (HH:MM:SS,mmm)
source_end	string	End timestamp in SRT format
context	string	Full context window text used for generation

Table 5.1: Ground truth schema field descriptions

5.1.2 Retrieval Metrics

Retrieval metrics evaluate the system's ability to find relevant chunks from the transcript before answer generation. Two complementary metrics are used: Hit Rate (recall-oriented) and Mean Reciprocal Rank (ranking-oriented).

5.1.2.1 Hit Rate Computation

Definition: Proportion of queries where at least one retrieved chunk has temporal overlap with the ground truth timestamp range [29]. The formula is shown in Figure 5.7:

```
Hit Rate = (Number of Hits) / (Total Queries)

where Hit = any retrieved chunk temporally overlaps with ground truth
```

Figure 5.7: Hit Rate formula for retrieval evaluation

Hit Rate is computed through temporal overlap detection between retrieved chunks and ground truth time ranges. Figure 5.8 shows the temporal overlap detection algorithm:

```python
def check_retrieval_relevance(
    retrieved_chunk_sources: List[Dict],
    ground_truth_start: str,
    ground_truth_end: str
) -> Tuple[bool, int]:

    gt_start = parse_timestamp(ground_truth_start)
    gt_end = parse_timestamp(ground_truth_end)

    for i, source in enumerate(retrieved_chunk_sources, 1):
        chunk_start = parse_timestamp(source['start_time'])
        chunk_end = parse_timestamp(source['end_time'])

        # Check for overlap
        # Overlap happens if chunk_start < gt_end AND chunk_end > gt_start
        if chunk_start < gt_end and chunk_end > gt_start:
            return True, i

    return False, 0
```

Figure 5.8: Interval intersection algorithm for retrieval relevance checking

The algorithm implements standard interval intersection logic: two intervals [a,b] and [c,d] overlap if and only if a $<$ d AND b $>$ c. For example, if a QA pair was generated from context spanning [10s, 20s] and a retrieved chunk covers [15s, 25s], the intervals overlap and the chunk is considered relevant. The function returns both a boolean hit flag and the 1-indexed rank of the first relevant chunk, enabling simultaneous Hit Rate and MRR computation.

5.1.2.2 Mean Reciprocal Rank (MRR)

Definition: Average of reciprocal ranks of the first relevant result, as illustrated in Figure 5.9 [9]:

```
MRR = (1 / |Q|) * Σ (1 / rank_i)

where:
  Q = set of queries
  rank_i = position of first relevant chunk for query i (1-indexed)
  rank_i = 0 if no relevant chunk found (contributes 0 to sum)
```

Figure 5.9: Mean Reciprocal Rank (MRR) formula and definition

If a hit is detected, the MRR contribution is computed as 1.0 divided by the rank. An MRR of 1.0 indicates perfect retrieval performance, where the first relevant result is consistently ranked at position 1. An MRR of 0.5 implies that the first relevant result appears at rank 2 on average, while an MRR of 0.333 suggests it appears at rank 3 on average. Conversely, an MRR of 0.0 indicates retrieval failure, where no relevant results are returned for the evaluated queries.

MRR provides a more nuanced assessment than Hit Rate by penalizing lower-ranked rele-

vant results. A system with Hit Rate = 1.0 and MRR = 0.5 successfully retrieves relevant chunks for all queries but often places them at rank 2 or lower, indicating suboptimal ranking quality.

5.1.3 Generation Metrics

Generation metrics evaluate the quality of answers produced by the LLM after retrieval. Two complementary metrics are used: F1 Score (lexical overlap) and LLM Judge Accuracy (semantic correctness).

5.1.3.1 F1 Score Calculation

Definition: Harmonic mean of precision and recall based on token-level overlap between generated and ground truth answers, as shown in Figure 5.10 [29]:

```
Precision = (Common Tokens) / (Predicted Tokens)
Recall = (Common Tokens) / (Ground Truth Tokens)
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

Figure 5.10: F1 score calculation with precision and recall

F1 Score computation requires token-level comparison between generated and ground truth answers. Figure 5.11 shows the normalization process:

```
def calculate_f1_score(prediction: str, ground_truth: str) -> float:
    """Calculates F1 score based on token overlap."""
    def normalize_text(text):
        return re.sub(r'\W+', ' ', text.lower()).strip()

    pred_tokens = normalize_text(prediction).split()
    gt_tokens = normalize_text(ground_truth).split()

    if not pred_tokens or not gt_tokens:
        return 0.0
```

Figure 5.11: Text preprocessing for token-level comparison

To enable robust token-level comparison, a lightweight normalization step is applied to both texts prior to metric computation. This process converts all characters to lowercase, removes punctuation and other non-word symbols via regex, and collapses repeated whitespace. This focuses the evaluation on lexical content rather than formatting differences.

Figure 5.12 shows the F1 computation using Counter intersection:

```
    common_tokens = collections.Counter(pred_tokens) & collections.Counter(gt_tokens)
    num_same = sum(common_tokens.values())

    if num_same == 0:
        return 0.0

    precision = num_same / len(pred_tokens)
    recall = num_same / len(gt_tokens)

    return 2 * (precision * recall) / (precision + recall)
```

Figure 5.12: Precision, recall, and harmonic mean calculation with Counter intersection

The Counter intersection operation returns the minimum count for each token present in both prediction and ground truth. For example, if the prediction contains ["apple", "apple", "banana"] and the ground truth contains ["apple", "banana", "banana"], the intersection yields {"apple": 1, "banana": 1}. The num_same variable sums these counts to get the total number of overlapping tokens.

Precision measures what fraction of the predicted tokens are correct. Recall measures what fraction of the ground truth tokens were predicted. F1 is the harmonic mean of precision and recall, providing a balanced measure that penalizes both over-generation (low precision) and under-generation (low recall).

The F1 score provides a balanced measure of lexical overlap between prediction and ground truth, but may underestimate semantic correctness when valid paraphrasing or synonym substitution is present. For example, "quarreled" vs "argued" would reduce F1 despite semantic equivalence.

5.1.3.2 LLM Judge Accuracy (End-to-End Accuracy)

Definition: Binary semantic correctness evaluated by an LLM judge, assessing whether the generated answer is semantically consistent with the ground truth [59]. Figure 5.13 shows the prompt template:

```
f"""Question: {question}
Ground Truth: {ground_truth}
Generated Answer: {prediction}

Is the Generated Answer semantically correct and consistent with the Ground Truth?
Respond with ONLY one word: "CORRECT" or "INCORRECT".

Response:"""
```

Figure 5.13: LLM judge prompt template for semantic correctness evaluation

The LLM judge receives a formatted prompt containing the question, ground truth answer, and generated prediction. The prompt explicitly instructs the judge to output only "CORRECT" or "INCORRECT" based on semantic consistency. The response is parsed using a hierarchical decision logic: if "CORRECT" appears without "INCORRECT", the

answer is marked correct; if "INCORRECT" appears, it is marked incorrect; otherwise, a fallback heuristic checks for "CORRECT" in the content. This handles edge cases where the judge provides additional explanation alongside the verdict.

The judge LLM uses temperature 0.0 for deterministic evaluation, reducing variability in scoring. Error handling returns False on judge failure, taking a conservative approach that prefers false negatives to false positives.

Both F1 and LLM Judge metrics are computed for each generated answer and accumulated for averaging. This dual evaluation provides complementary perspectives: F1 captures lexical fidelity while LLM Judge captures semantic correctness.

5.1.4 Separation of Retrieval and Generation Evaluation

The evaluation framework deliberately separates retrieval and generation metrics to enable diagnostic analysis:

Computed Before Generation: Hit Rate and MRR. These metrics do not depend on the generated answer and evaluate only the retrieval pipeline's ability to find relevant chunks.

Computed After Generation: F1 Score and LLM Judge. These metrics evaluate the LLM's ability to extract and synthesize information from the retrieved context, holding the retrieved chunks constant.

This separation enables precise failure attribution. Figure 5.14 illustrates Scenario 1 (retrieval failure):

Scenario 1: Retrieval Failure

```
{
  "retrieval_hit": false,
  "retrieval_rank": 0,
  "is_correct": false,
  "f1_score": 0.0
}
```

Figure 5.14: Retrieval failure scenario diagnostic example

Retrieval pipeline failed to find relevant chunks. One example action is to increase the retriever's top-k value and re-evaluate performance, in combination with adjusting fusion weights and/or refining the reranker configuration.

Figure 5.15 shows Scenario 2 (generation failure):

Scenario 2: Generation Failure

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.42
}
```

Figure 5.15: Generation failure scenario with successful retrieval

Retrieval succeeded, but the LLM failed to extract the correct answer from the retrieved context. One example action is to refine the prompt (e.g., by adding explicit extraction instructions or structured output formats); additionally, we can experiment with alternative LLMs and temperature settings to improve answer reliability.

Figure 5.16 illustrates Scenario 3 (perfect performance):

Scenario 3: Perfect Performance

```
{
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.87
}
```

Figure 5.16: Perfect performance scenario for both retrieval and generation

Both retrieval and generation working optimally.

5.1.5 Speed Evaluation Pipeline

This section introduces a lightweight speed evaluation pipeline that is separated from the main evaluation pipeline to enable an unbiased latency comparison between the RAG-based approach and a Pure LLM baseline. The pipeline reuses the same ground truth dataset described in Section 5.1.1, including identical questions, reference answers, and temporal metadata, to ensure consistency across evaluations. This separation is necessary due to prompt caching effects in modern LLM services, which can artificially distort latency measurements under sequential execution [60]. The mitigation strategy adopted to address this issue is discussed in detail in Section 5.1.5.1.

5.1.5.1 Interleaved Execution Design

The speed evaluation pipeline addresses prompt caching bias through interleaved execution.

LLM services cache recently seen prompts to improve efficiency [60]. A sequential benchmarking setup could unintentionally allow the Pure LLM baseline to benefit from warmed

caches, thereby underestimating its true latency and producing unfair comparisons. To address this, the pipeline alternates between RAG and Pure LLM execution on a per-question basis, ensuring both methods experience similar caching conditions and enabling a more reliable and unbiased measurement of relative runtime performance, as illustrated in Figure 5.17.

```
Query 1: RAG → Pure LLM
Query 2: RAG → Pure LLM
Query 3: RAG → Pure LLM
...
```

Figure 5.17: Interleaved execution pattern for unbiased speed benchmarking

5.1.5.2 Warmup Phase

The evaluation pipeline includes a warmup phase, which is designed to reduce cold-start effects and improve the reliability of latency benchmarking. Specifically, it triggers just-in-time (JIT) compilation and warms up the Python runtime and dependent libraries, initializes and primes the LLM, embedding, and reranker services, and preloads model weights into GPU memory to avoid first-run loading overhead. In addition, it establishes and stabilizes HTTP connections to mitigate transient network latency and connection setup costs during the measured runs.

Warmup results are excluded from timing statistics to measure steady-state performance.

5.1.6 Statistical Computation

5.1.6.1 Basic Statistics

For each approach (RAG and Pure LLM), the following statistics are computed, as shown in Table 5.2:

Metric	Description
Mean	Average query time across all timed queries
Median	Middle value (robust to outliers)
Min	Best-case performance (fastest query)
Max	Worst-case performance (slowest query)
Std Dev	Standard deviation (consistency measure)

Table 5.2: Statistical metrics table for latency analysis

5.1.6.3 Comparative Metrics

Direct comparison between RAG and Pure LLM is quantified through several metrics, as illustrated in Table 5.3:

Metric	Formula	Interpretation
Speedup Factor	Pure LLM Mean / RAG Mean	>1: RAG faster, <1: Pure LLM faster, =1: equivalent
Time Difference (s)	Pure LLM Mean - RAG Mean	Absolute time saved per query
Percentage Faster	(Difference / Pure LLM Mean) × 100	Relative improvement percentage

Table 5.3: Comparative metrics between RAG and Pure LLM approaches

5.2 Experimental Results

This section presents the empirical results from evaluating the RAG system across four diverse transcripts, covering both quality metrics (retrieval and generation performance) and speed performance (latency comparison with Pure LLM baseline).

It should be noted that the results reported in this section are not final, as the experiments were conducted using a locally hosted LLM for development purpose, which is significantly smaller than the self-hosted LLM deployed on the Meadow9+ production server. Consequently, the absolute latency and generation performance observed here may not directly reflect production-level behavior, and further benchmarking on the target deployment environment is required for conclusive comparison.

5.2.1 RAG Quality Metrics

5.2.1.1 Per-Transcript Results

Table 5.4 presents the per-transcript RAG quality metrics:

Transcript	Duration	Questions	Hit Rate	MRR	Accuracy	F1 Score
Transcript 1	00:08:21	9	1.000	1.000	0.889	0.677
Transcript 2	01:59:40	30	0.867	0.867	0.600	0.476
Transcript 3	00:08:14	8	1.000	1.000	1.000	0.613
Transcript 4	00:36:14	15	1.000	1.000	0.667	0.446
Aggregate	**02:52:29**	**62**	**0.967**	**0.967**	**0.789**	**0.553**

Table 5.4: Per-transcript RAG quality metrics aggregated results

5.2.1.2 Aggregate Interpretation

Retrieval Performance:

- **Hit Rate: 96.7%:** 60 out of 62 queries successfully retrieved at least one temporally relevant chunk

- **MRR: 0.967:** The high average reciprocal rank indicates that relevant chunks typically appear at rank 1

The near-perfect retrieval metrics validate the hybrid search architecture. Only Transcript 2 (the longest at almost 2 hours) exhibited any retrieval failures, with 4 out of 30 queries missing relevant chunks.

Generation Performance:

- **Accuracy: 78.9%:** LLM Judge marked 49 out of 62 generated answers as semantically correct
- **F1 Score: 0.553:** Moderate token-level overlap between generated and ground truth answers

The divergence between Accuracy and F1 (0.789 vs 0.553) indicates that the LLM frequently paraphrases or synthesizes information rather than copying text verbatim from retrieved chunks. This is desirable behavior, as it demonstrates understanding rather than mere extraction.

5.2.1.4 Detailed Example: Success Case

Figure 5.18 shows a detailed success case:


```
{
  "question": "According to the passage, what did the North Wind and the
               Sun quarrel about?",
  "ground_truth": "The North Wind and the Sun quarreled about which of
                   them was the stronger.",
  "generated_answer": "According to the passage, the North Wind and the
                       Sun quarrelled about which of them was the
                       stronger.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": true,
  "f1_score": 0.8125,
  "sources": [
    {
      "chunk_id": 1,
      "start_time": "00:00:15,140",
      "end_time": "00:01:31,240",
      "rerank_score": 3.172,
      "preview": "the north wind and the sun had a quarrel about which of
                  them was the stronger..."
    }
  ]
}
```


Figure 5.18: Detailed success case example with perfect retrieval and generation

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1 with high rerank score (3.172)
- **Near-perfect generation:** Minor spelling variation ("quarreled" vs "quarrelled") does not affect semantic correctness
- **High F1 (0.8125):** Strong lexical overlap due to minimal paraphrasing
- **LLM Judge:** Correct: Semantic equivalence recognized despite spelling difference

This represents optimal system performance across all stages.

5.2.1.5 Detailed Example: Failure Case

Figure 5.19 shows a failure case:

```
{
  "question": "What does the speaker find unusual about the blind box packaging?",
  "ground_truth": "The speaker notes that the box does not say 'Laboo' or
                   'Labu' anywhere, but instead says 'exciting macaron' and
                   'the monsters', and that the top sticker feels tampered
                   with.",
  "generated_answer": "Based on the provided excerpts, the speaker finds that
                       the packaging of the blind box \"smells kind of
                       artificial\" and specifically \"like a carpet\"
                       (Chunk 2).",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.291
}
```

Figure 5.19: Failure case example with retrieval success but generation error

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1
- **Generation failure:** Model focused on smell observations instead of labeling/branding issues
- **Low F1 (0.291):** Minimal lexical overlap - completely different aspects mentioned
- **LLM Judge:** Incorrect: Answer addresses wrong aspect of "unusual"

This is a true generation failure where retrieval succeeded in finding relevant context, but the LLM selected less salient information (smell) over the main point (missing brand name). The model may have latched onto sensory details in the retrieved chunk rather than the more significant observation about brand labeling. Such errors may be mitigated through prompt engineering that explicitly emphasizes answering the core question

over incidental attributes, context window optimization to reduce distracting noise, fine-tuning on examples that require distinguishing primary from secondary information, and lightweight post-processing checks to validate that the generated answer aligns with the intended focus of the query.

5.2.1.6 Detailed Example: False Negative

Figure 5.20 illustrates a false negative example:

```
{
  "question": "What was the hardest decision the speaker had to make?",
  "ground_truth": "The hardest decision was choosing between two job offers:
                   one with NUHS in the memory clinic doing research, and the
                   other at the current place, which they ultimately chose.",
  "generated_answer": "The hardest decision the speaker had to make was
                       choosing between two job offers. One was with a memory
                       clinic doing research, and the other was joining HDB.
                       The speaker had to make this decision within a few days,
                       and in the end, they chose to join HDB, which they are
                       happy with.",
  "retrieval_hit": true,
  "retrieval_rank": 1,
  "is_correct": false,
  "f1_score": 0.585
}
```

Figure 5.20: False negative example with more specific generated answer

Analysis:

- **Perfect retrieval:** Relevant chunk ranked at position 1
- **More specific generation:** Generated answer mentions "HDB" explicitly, while ground truth uses vague "current place"
- **Moderate F1 (0.585):** Partial lexical overlap captures the correct structure with added specificity
- **LLM Judge:** Incorrect: Marked as incorrect due to specificity mismatch

This represents a false negative where the generated answer is actually more accurate and informative than the ground truth. The transcript does mention HDB (Housing & Development Board), making this not a hallucination but rather additional specificity. The LLM Judge may have been overly strict in requiring exact phrasing match. This example highlights a common evaluation challenge when model outputs include correct extra details, where vague ground truth annotations (e.g., “current place”) can inadvertently penalize semantically equivalent but more specific answers. To address this, evaluation should incorporate metrics and judging criteria that tolerate acceptable specificity variation, and the ground truth and/or judge prompt can be revised to recognize paraphrases and grounded elaborations as correct.

From observation, such cases are relatively frequent in practice, where correct generations are incorrectly marked as wrong, which may lead to an underestimation of the true accuracy of the RAG pipeline. This issue is planned to be addressed within the current semester through refinements to the evaluation methodology, and the updated approach will be documented in the final report.

5.2.2 Speed Performance

5.2.2.1 Per-Transcript Speed Comparison

Table 5.5 presents the per-transcript speed comparison:

Transcript	Duration	RAG Mean (s)	Pure LLM Mean (s)	Speedup Factor
Transcript 1	00:08:21	7.01	10.03	1.43x
Transcript 2	01:59:40	7.39	51.62	6.99x
Transcript 3	00:08:14	6.70	11.57	1.73x
Transcript 4	00:36:14	7.14	49.63	6.95x
Aggregate	**02:52:29**	**7.15**	**38.53**	**5.39x**

Table 5.5: Per-transcript speed comparison showing speedup factors

5.2.2.2 Speedup vs Transcript Size Analysis

RAG Performance (Stable):

- Mean RAG time ranges from 6.70s to 7.39s across all transcripts
- Retrieval extracts a small, fixed-size context regardless of transcript length
- Generation time depends only on retrieved context size, not source transcript size
- Result: Consistent latency independent of transcript length

Pure LLM Performance (Scales with Size):

- Generation time scales approximately linearly with context size
- Result: Latency grows with transcript length

RAG provides greater speed advantages for longer transcripts, which are common in real-world meeting recording scenarios.

5.2.2.5 Quality vs Speed Trade-Off Analysis

Table 5.6 summarizes the quality-speed trade-off analysis:

Transcript	**Speedup**	**RAG Accuracy**	**Pure LLM Accuracy**	**Accuracy Δ**	**Trade-Off**
Transcript 1	1.43x	0.889	0.875	+0.014	Speed + Quality
Transcript 2	6.99x	0.600	0.667	-0.067	Speed » Quality
Transcript 3	1.73x	1.000	1.000	0.000	Speed + Quality
Transcript 4	6.95x	0.667	0.733	-0.066	Speed » Quality

Table 5.6: Quality-speed trade-off analysis across transcript sizes

RAG provides substantial speed improvements (1.4x to 7x) with either comparable or slightly reduced quality. For small transcripts, RAG is strictly superior (faster and equal/better quality). For large transcripts, RAG offers a favorable trade-off with massive speedup for small accuracy reduction.

Chapter 6

Conclusion and Future Work

This chapter will be completed in the final report, once everything has been finalized.

6.1 Future Work

Based on consultations with project supervisors, several areas for future development have been identified.

6.1.1 Limited Coverage of Question Types in the Evaluation Pipeline

The current evaluation pipeline is designed under the assumption that users ask highly specific questions that require extracting precise, localized information from a transcript. This is reflected in the construction of the ground truth dataset, where questions are intentionally scoped to narrow spans of evidence. Examples include:

- "What is the example given of a summarized headline about Baldur's Gate 3 players?"
- "What is the rarity of the secret character in the blind box compared to the other characters?"
- "What does the speaker find unusual about the blind box packaging?"

However, the evaluation pipeline does not yet cover broader and more open-ended queries such as "What is being discussed in this meeting?", where the answer may require integrating context across multiple topics and long time ranges [5]. Additional examples of such vague queries include:

- "Can you summarize the main points from this meeting?"
- "What are the key decisions made in this conversation?"
- "What problems or concerns were raised by the team?"
- "What action items or next steps were mentioned?"
- "What is the overall tone or sentiment of the discussion?"

These question types may perform worse under the current retrieval design, as they often require global transcript-level understanding rather than a small set of relevant chunks [5].

Extending the evaluation pipeline to include these high-level summarization and meeting-understanding queries will be addressed in the current semester and incorporated into the final report.

6.1.2 RAG Pipeline Optimization for Ambiguous Queries

The existing RAG pipeline requires enhancement to better handle vague or underspecified questions as described in Section 6.1.1. Proposed improvements include:

- Integration of document summaries into the LLM context to provide high-level contextual information
- Incorporation of neighboring retrieved chunks to maintain semantic coherence across chunk boundaries
- Leveraging the expanded context window available in the Meadow9+ server deployment, which utilizes a larger LLM compared to the locally hosted variant used for development

These enhancements necessitate comprehensive re-evaluation of system latency to ensure performance requirements remain satisfied despite the increased context size [6].

6.1.3 Production Deployment of Retrieval Server

Subject to time constraints, investigation into full production deployment of the retrieval server infrastructure will be undertaken. This includes:

- Infrastructure provisioning and scalability considerations
- Deployment architecture design
- Production monitoring and maintenance protocols

Bibliography

[1] M. Shao, A. Basit, R. Karri, and M. Shafique, "Survey of different large language model architectures: Trends, benchmarks, and challenges," *IEEE Access*, vol. 12, pp. 188 664–188 706, 2024.

[2] Hostinger. (2025) Llm statistics 2025: Adoption, trends, and market insights. [Online]. Available: https://www.hostinger.com/tutorials/llm-statistics

[3] S. K. Dam, N. Sharma, M. Sahay, and M. Shah, "A complete survey on llm-based ai chatbots," *arXiv preprint arXiv:2406.16937*, 2024.

[4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[6] NVIDIA. (2023) Mastering llm techniques: Inference optimization. NVIDIA Technical Blog. [Online]. Available: https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/

[7] K. Kuhn, V. Kersken, B. Reuter, N. Egger, and G. Zimmermann, "Measuring the accuracy of automatic speech recognition solutions," *ACM Transactions on Accessible Computing*, vol. 16, no. 4, pp. 25:1–25:23, 2023.

[8] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2024.

[9] Wikipedia. (2024) Mean reciprocal rank. [Online]. Available: https://en.wikipedia.org/wiki/Mean_reciprocal_rank

[10] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.

[11] R. Nogueira and K. Cho, "Passage re-ranking with bert," *arXiv preprint arXiv:1901.04085*, 2019.

[12] Otter.ai. (2026) Otter transcribe voice notes. [Online]. Available: https://apps.apple.com/na/app/otter-transcribe-voice-notes/id1276437113

[13] ——. (2026) Ai sales notetaker. [Online]. Available: https://otter.ai/sales-agent

[14] ——. (2026) Ai education notetaker for real-time lecture notes. [Online]. Available: https://otter.ai/education-agent

[15] ——. (2026) Otter features. [Online]. Available: https://otter.ai/features

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] Otter.ai Help Center. (2026) Otter AI chat FAQs. [Online]. Available: https://help.otter.ai/hc/en-us/articles/17016733191703-Otter-AI-Chat-FAQs

[18] Anthropic. (2026) Claude code tutorials. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/tutorials

[19] ——. (2026) Claude code settings. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/settings

[20] ——. (2026) Claude code team. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/team

[21] ——. (2026) Manage claude code memory. [Online]. Available: https://docs.anthropic.com/en/docs/claude-code/memory

[22] ——. (2026) Using extended thinking. [Online]. Available: https://support.anthropic.com/en/articles/10574485-using-extended-thinking

[23] Meadow9. (2026) Meadow9. [Online]. Available: https://apps.apple.com/au/app/meadow9/id1474941556

[24] ABAX.AI. (2026) Abax ai | custom speech to text. [Online]. Available: https://www.abax.ai/

[25] ABAX.AI PRIVATE LIMITED. (2026) Abax.ai private limited (202128671g). [Online]. Available: https://www.companies.sg/business/202128671G/ABAX-AI-PRIVATE-LIMITED

[26] Meadow9+. (2026) Meadow9+. [Online]. Available: https://apps.apple.com/us/app/meadow9/id6740014005

[27] P. E. Black. (2021) double metaphone. National Institute of Standards and Technology. [Online]. Available: https://www.nist.gov/dads/HTML/doubleMetaphone.html

[28] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Dokl. Akad. Nauk SSSR*, vol. 163, no. 4, pp. 845–848, 1965. [Online]. Available: https://www.mathnet.ru/eng/dan31411

[29] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979. [Online]. Available: https://search.worldcat.org/title/Information-retrieval/oclc/4494554

[30] LangChain. (2026) Document. [Online]. Available: https://api.python.langchain.com/en/latest/core/documents/langchain_core.documents.base.Document.html

[31] ——. (2026) Prompt templates. [Online]. Available: https://python.langchain.com/docs/concepts/prompt_templates/

[32] ——. (2026) Output parsers. [Online]. Available: https://api.python.langchain.com/en/latest/core/output_parsers.html

[33] FastAPI. (2026) Fastapi. [Online]. Available: https://fastapi.tiangolo.com/

[34] Pydantic. (2026) Pydantic. [Online]. Available: https://docs.pydantic.dev/

[35] Jina AI. (2026) jina-embeddings-v3. [Online]. Available: https://huggingface.co/jinaai/jina-embeddings-v3

[36] BAAI. (2026) bge-reranker-v2-m3. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3

[37] Qwen Team. (2026) Qwen3-14b-base. [Online]. Available: https://huggingface.co/Qwen/Qwen3-14B-Base

[38] DeepSeek. (2026) Deepseek api documentation. [Online]. Available: https://api-docs.deepseek.com/

[39] ggml-org. (2026) llama.cpp. [Online]. Available: https://github.com/ggml-org/llama.cpp

[40] LangChain. (2026) Openai embeddings. [Online]. Available: https://python.langchain.com/docs/integrations/text_embedding/openai/

[41] ——. (2026) Chroma vector store. [Online]. Available: https://python.langchain.com/api_reference/chroma/vectorstores/langchain_chroma.vectorstores.Chroma.html

[42] Chroma. (2026) Performance. [Online]. Available: https://docs.trychroma.com/guides/deploy/performance

[43] rank-bm25. (2026) rank-bm25. [Online]. Available: https://pypi.org/project/rank-bm25/

[44] metaphone. (2026) metaphone. [Online]. Available: https://pypi.org/project/metaphone/

[45] RapidFuzz. (2026) Rapidfuzz usage: fuzz. [Online]. Available: https://rapidfuzz.github.io/RapidFuzz/Usage/fuzz.html

[46] wordfreq. (2026) wordfreq. [Online]. Available: https://pypi.org/project/wordfreq/

[47] dateparser. (2026) dateparser documentation. [Online]. Available: https://dateparser.readthedocs.io/en/stable/index.html

[48] pandas. (2026) pandas documentation. [Online]. Available: https://pandas.pydata.org/docs/dev/index.html

[49] NumPy. (2026) Numpy documentation. [Online]. Available: https://numpy.org/doc/2.0/

[50] Matplotlib. (2026) Matplotlib documentation. [Online]. Available: https://matplotlib.org/stable/

[51] pytest. (2026) pytest documentation. [Online]. Available: https://docs.pytest.org/en/9.0.x/

[52] Requests. (2026) Requests: Http for humans. [Online]. Available: https://docs.python-requests.org/en/latest/

[53] OpenAI. (2026) Embeddings. [Online]. Available: https://platform.openai.com/docs/api-reference/embeddings

[54] LangChain. (2026) Chatprompttemplate. [Online]. Available: https://python.langchain.com/api_reference/core/prompts/langchain_core.prompts.chat.ChatPromptTemplate.html

[55] Qwen Team. (2026) Qwen3. [Online]. Available: https://github.com/QwenLM/Qwen3

[56] LangChain. (2026) Messagesplaceholder. [Online]. Available: https://api.python.langchain.com/en/latest/prompts/langchain_core.prompts.chat.MessagesPlaceholder.html

[57] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "Ares: An automated evaluation framework for retrieval-augmented generation systems," *arXiv preprint arXiv:2311.09476*, 2023.

[58] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.

[59] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: Nlg evaluation using gpt-4 with better human alignment," *arXiv preprint arXiv:2303.16634*, 2023.

[60] OpenAI. (2024) Prompt caching in the api. [Online]. Available: https://openai.com/index/api-prompt-caching/